

Trabajo Fin de Máster

SmartDeliveryAI

*Plataforma inteligente para la predicción y gestión de entregas logísticas mediante
Machine Learning y sistemas multiagente*

Máster en Big Data, Data Science e Inteligencia Artificial

Integrantes:

Aldana Lazón Díaz
Andrés López Sierra
Juan Sebastián Galvis Chacón
Marcos Barba Ballarin
Tania de Mingo Frade

Tutores: Carlos Ortega y Santiago Mota
Fecha de entrega: 17 de septiembre 2026

Índice

Contenido

1. Resumen ejecutivo	1
2. Contexto y objetivo del proyecto	1
2.1 Contexto del sector logístico.....	1
2.2 Objetivo principal	2
2.3 Objetivos secundarios	2
2.4 Alcance y exclusiones	2
3. Datos utilizados.....	2
3.1 Fuente y derechos de uso	2
3.2 Consideraciones de protección de datos	3
3.3 Variables principales	3
3.4 Volumen, periodo y representatividad.....	3
4. Arquitectura técnica y pipeline de datos	3
4.1 Capas Bronze, Silver y Gold.....	3
4.2 Las dos tablas Gold.....	4
4.3 Reproducibilidad y gobierno de datos.....	4
5. Análisis exploratorio (EDA)	4
5.1 La variable objetivo son en realidad dos poblaciones	5
5.2 Un baseline de referencia exigente y contraintuitivo	5
5.3 La distancia no lo explica todo.....	5
5.4 El clima no aporta y además engaña.....	5
5.5 Valores atípicos: no se recortan	6
5.6 Contexto geográfico y temporal	6
6. Preparación y transformación de datos	6
6.1 Limpieza.....	6
6.2 Ingeniería de variables	6
6.3 Separación por población y codificación.....	6
6.4 Estrategia de partición: GroupKFold por ruta.....	7
6.5 La regla de no fuga de datos	7
7. Modelos desarrollados	7
7.1 Arquitecturas evaluadas	7
7.2 Validación.....	9
8. Resultados y comparativa	9
8.1 Comparativa de modelos.....	9
8.2 Análisis de errores fuera de muestra	10
8.3 Sesgo sistemático y su mecanismo.....	10
8.4 El error no se concentra en una estación, pero el sesgo sí es unánime	11
8.5 El error no empeora significativamente a lo largo de la ruta	11
8.6 Distribución del residuo y casos peor predichos	12

8.7 Del error por tramo al error por ruta	12
9. Interpretabilidad	13
9.1 Importancia global de variables — tramos de reparto	13
9.2 Importancia global de variables — tramo de salida del almacén.....	14
9.3 Riesgos y sesgos a comunicar	15
10. Productivización.....	15
10.1 De un modelo entrenado a un producto usable	15
10.2 El sistema multiagente.....	16
10.3 El servidor de herramientas (MCP).....	17
10.4 La aplicación	17
11. Conclusiones	17
11.1 Conclusiones de negocio.....	17
11.2 Conclusiones técnicas	18
11.3 Limitaciones	18
11.4 Líneas futuras.....	18
12. Bibliografía y referencias.....	18
Anexo A · Detalle de la arquitectura y el pipeline de datos.....	20
A.1 Descarga del dataset de Amazon	20
A.1.1 Qué se ha hecho.....	20
A.1.2 Por qué se ha elegido este dataset	20
A.1.3 Dónde se ha guardado.....	20
A.1.4 Archivos descargados	20
A.1.5 Verificación realizada	21
A.1.6 Reproducibilidad	21
A.1.7 Protección para GitHub	21
A.2 Creación de Bronze con fuentes externas fiables	22
A.2.1 Qué se ha hecho.....	22
A.2.2 Por qué se ha hecho	22
A.2.3 Fuentes añadidas.....	22
A.2.4 Cómo se ha implementado	23
A.2.5 Verificación realizada	23
A.2.6 Cómo se usará más adelante.....	23
A.2.7 Por qué no se ha hecho en Silver directamente	24
A.3 Creación de Silver	24
A.3.1 Qué se ha hecho.....	24
A.3.2 Por qué se ha hecho	24
A.3.3 Cómo se ha implementado	25
A.3.4 Tablas Silver generadas.....	25
A.3.5 Resultados obtenidos.....	25
A.3.6 Decisión sobre motor y formato.....	26
A.3.7 Uso previsto en la fase siguiente.....	26

A.4 Creación de Gold: la tabla de rutas.....	26
A.4.1 Qué se ha hecho.....	26
A.4.2 Por qué se ha hecho	27
A.4.3 Cómo se ha implementado	27
A.4.4 Dataset Gold generado	27
A.4.5 Validación realizada	28
A.4.6 Uso previsto en el EDA	28
A.5 Pipeline completo	28
A.5.1 Qué se ha hecho.....	28
A.5.2 Para qué sirve	29
A.5.3 Qué hace	29
A.5.4 Resultado de la ejecución	29
A.6 Entorno de ejecución multiplataforma (Windows)	29
A.6.1 Qué se ha hecho.....	29
A.6.2 Por qué se ha hecho	29
A.6.3 Cómo se ha implementado	30
A.6.4 Resultado.....	30
A.6.5 Dependencias fijadas y reproducibilidad	30
A.6.6 Reproducibilidad para otros equipos	30
A.7 Distancia recorrida por ruta en Gold	30
A.7.1 Qué se ha hecho.....	30
A.7.2 Por qué se ha hecho	31
A.7.3 Hallazgo: el trayecto al barrio pesa el 38% de la distancia	31
A.7.4 Cómo se ha implementado	31
A.7.5 Resultado.....	32
A.7.6 Continuidad del trabajo	32
A.8 Construcción de la tabla de tramos (Dataset Tramos Ruta).....	32
A.8.1 Qué se ha hecho.....	32
A.8.2 Por qué se ha hecho	33
A.8.3 De dónde sale la variable objetivo.....	33
A.8.4 Cómo se ha implementado	34
A.8.5 Variables generadas y control de fuga	34
Anexo B · Código de los modelos ganadores	35
B.1 La variable que decide el modelo ganador (04 CatBoost.py)	35
B.2 Hiperparámetros de los dos especialistas (06 SHAP.py).....	36
B.3 Interpretabilidad: TreeSHAP nativo (06 SHAP.py).....	36
B.4 Predicción fuera de muestra y control de fuga (07 Análisis de Errores.py).....	37
B.5 Persistencia y carga del modelo aprobado (06 SHAP.py)	37

1. Resumen ejecutivo

En el reparto de última milla, saber a qué hora va a llegar un pedido no es un detalle menor sino que condiciona la experiencia del cliente, el coste operativo de las incidencias y la capacidad de una empresa logística para planificar su flota. Este Trabajo Fin de Máster desarrolla SmartDeliveryAI, un sistema de predicción del tiempo de entrega en reparto de última milla, junto con un prototipo de productivización basado en agentes inteligentes que integra esa predicción en la operación diaria de una empresa de reparto.

El proyecto parte de un dataset real y público: el Amazon Last Mile Routing Research Challenge, con 6.112 rutas históricas de reparto realizadas por conductores de Amazon en 2018 en cinco áreas metropolitanas de Estados Unidos, enriquecido con datos meteorológicos históricos (Open-Meteo) y calendario de festivos (Nager.Date). Sobre este dataset se construyó una arquitectura de datos por capas (Bronze, Silver y Gold) con PySpark, y se entrenó un conjunto de modelos de Machine Learning para predecir el tiempo de conducción entre cada dos paradas consecutivas de una ruta, la magnitud que, sumada, permite estimar la hora de llegada a cualquier domicilio.

El resultado principal es un modelo híbrido (CatBoost para tramos de reparto, LightGBM para el tramo de salida del almacén) que reduce el error medio de predicción a 0,3721 minutos por tramo, una mejora del 16,8% sobre un baseline exigente de referencia (MAE 0,4473 min, R^2 0,919), superando también a una red LSTM entrenada como alternativa de deep learning. El modelo se ha auditado con un análisis de errores fuera de muestra e interpretado con SHAP, identificando la distancia del tramo y la estación de origen como los factores que más pesan en cada predicción, y documentando de forma explícita un sesgo sistemático de infrapredicción del 10,1% en los tramos de reparto, más acusado cuanto más largo es el tramo.

Más allá de la métrica, el proyecto pone un foco deliberado en dos aspectos que la normativa del máster señala como diferenciales de un TFM frente a un ejercicio de asignatura: la interpretabilidad del modelo, imprescindible para poder explicar a un cliente por qué se retrasa su pedido, y la productivización, es decir, cómo este modelo deja de ser un notebook y pasa a ser un servicio que una empresa de reparto puede usar. Esta segunda parte no se ha quedado en el diseño: se ha construido un prototipo funcional, formado por tres agentes especializados (supervisión, optimización de rutas, y cálculo y explicación de la hora de llegada), un servidor de herramientas que mantiene los modelos cargados y una aplicación web desde la que se maneja todo, tal y como se detalla en la sección 10.

El valor de negocio esperado es doble: de cara al cliente final, una estimación de entrega más ajustada y explicable reduce la incertidumbre y las llamadas de soporte; de cara a la operación, detectar con antelación qué pedidos tienen mayor riesgo de retraso permite intervenir antes de que el retraso se produzca.

2. Contexto y objetivo del proyecto

2.1 Contexto del sector logístico

El reparto de última milla (el tramo final que lleva un paquete desde el centro logístico hasta la puerta del cliente) concentra buena parte del coste y de la insatisfacción en la cadena de distribución de comercio electrónico. Un tiempo de entrega mal estimado tiene dos costes distintos: para el cliente, la incertidumbre de no saber cuándo va a llegar su pedido; para la empresa, el coste de gestionar reclamaciones, reprogramar entregas fallidas y no poder anticipar cuellos de botella en la operación.

La mayoría de las plataformas de reparto actuales ofrecen ventanas horarias amplias y poco ajustadas, o estimaciones que no se explican: el cliente recibe una hora, pero no una razón. SmartDeliveryAI aborda ambos problemas a la vez: una predicción más precisa del tiempo de entrega, y un modelo interpretable que permite construir explicaciones honestas ("su pedido se retrasa porque la ruta de hoy tiene más paradas de lo habitual", no una excusa genérica).

2.2 Objetivo principal

Predecir con precisión el tiempo de conducción entre dos paradas consecutivas de una ruta de reparto, de forma que, sumando tramos, se pueda estimar la hora de llegada a cualquier parada de la ruta a partir de información conocida antes de que el vehículo salga del almacén.

2.3 Objetivos secundarios

- Comparar de forma rigurosa varias familias de modelos (lineal, árboles de gradiente, híbridos por población, deep learning secuencial) frente a un baseline exigente, evitando presentar una mejora ficticia.
- Explicar el comportamiento del modelo con técnicas de interpretabilidad (SHAP), de forma que sus predicciones puedan justificarse ante un cliente o un responsable de operaciones.
- Construir un prototipo de productivización que muestre cómo este modelo se integra en una aplicación empresarial real, con una capa de agentes que traduce la predicción en respuestas útiles para distintos usuarios (cliente, operaciones).
- Documentar de forma explícita las limitaciones del dataset y del enfoque, en línea con lo que la normativa del máster pide a un TFM: no ocultar los límites de los datos, sino analizarlos.

2.4 Alcance y exclusiones

El proyecto predice tiempos de conducción a partir de información planificada (distancia, zona, carga de la parada, hora de salida, meteorología prevista), no de señales en tiempo real. Quedan explícitamente fuera del alcance: el tráfico en tiempo real (el dataset no lo incluye y las coordenadas están ofuscadas, lo que impide cruzarlas con fuentes externas de tráfico), el routing por carretera real (las distancias se calculan en línea recta y se documenta esa limitación), y la predicción de qué entregas van a fallar por ausencia del destinatario, ya que el dataset no contiene ningún historial del cliente que permita anticiparlo. El sistema sí está pensado para reaccionar ante un fallo ya registrado, recalculando el resto de la ruta.

3. Datos utilizados

3.1 Fuente y derechos de uso

La fuente principal es el Amazon Last Mile Routing Research Challenge, publicado por Amazon en el registro AWS Open Data. Amazon describe el conjunto completo del reto como datos reales de rutas de última milla realizadas por conductores en 2018 en cinco áreas metropolitanas de Estados Unidos, con información a nivel de ruta, parada y paquete, e identificadores anonimizados. El acceso es público, vía bucket S3, sin necesidad de cuenta AWS.

El reto distribuye los datos en varios subconjuntos; este proyecto trabaja sobre el conjunto de entrenamiento, que es el único que incluye la secuencia real de entrega ejecutada por el conductor y que reúne 6.112 rutas. La licencia es Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0), compatible con un uso académico y no comercial como este TFM.

Se ha enriquecido con dos fuentes externas, ambas de acceso abierto y sin necesidad de clave de API: meteorología histórica diaria por estación logística (Open-Meteo Historical Weather API,

licencia CC BY 4.0, que exige atribución visible) y calendario de festivos oficiales de Estados Unidos para 2018 (Nager.Date Public Holiday API, uso libre).

3.2 Consideraciones de protección de datos

El dataset viene anonimizado de origen, es decir, los identificadores de ruta, parada y paquete son hashes, y las coordenadas de los puntos de entrega están deliberadamente ofuscadas por Amazon. No se ha identificado ningún dato personal en el conjunto (nombres, direcciones exactas o datos de contacto de clientes), por lo que no aplican las obligaciones específicas del RGPD sobre datos de carácter personal. Esta valoración se ha revisado de forma explícita como parte de la selección del dataset, siguiendo la recomendación de la normativa del máster de justificar los derechos de uso y las implicaciones de protección de datos dentro del propio trabajo.

3.3 Variables principales

Variable	Origen	Significado
route_id / stop_id	Rutas Amazon	Identificadores de ruta y de parada dentro de la ruta.
station_code	Rutas Amazon	Estación logística (almacén) de la que sale la ruta.
stops.lat / lng, zone_id	Rutas Amazon	Coordenadas y zona de planificación de cada parada.
actual (secuencia real)	Secuencias reales	Orden real en el que el conductor visitó las paradas.
travel_times (matriz)	Tiempos de viaje	Tiempo histórico entre cada par de paradas de la ruta; de aquí sale la variable objetivo.
scan_status, dimensiones, ventana horaria	Paquetes Amazon	Estado de entrega y características de cada paquete en cada parada.
temperature, rain_sum, wind_speed...	Open-Meteo	Meteorología diaria por estación logística.
is_us_public_holiday, weekday	Nager.Date + calendario	Contexto de calendario del día de la ruta.

Tabla 1. Variables principales del dataset, agrupadas por fuente de origen.

3.4 Volumen, periodo y representatividad

El conjunto de entrenamiento reúne 6.112 rutas, con 904.527 paradas y 898.415 tramos entre paradas consecutivas (la unidad que finalmente alimenta el modelo), repartidas entre 17 estaciones logísticas de cinco áreas metropolitanas. El periodo cubierto va del 19 de julio al 26 de agosto de 2018, cinco semanas consecutivas de verano. Esta ventana temporal es representativa del volumen de operación diaria de un centro logístico real, pero limita el análisis de estacionalidad: no hay datos de invierno, no cae ningún festivo oficial en el periodo, y por tanto el dataset no permite evaluar cómo cambia el reparto en fechas señaladas. Esta y otras limitaciones del dataset se analizan en detalle en la sección 11.3.

4. Arquitectura técnica y pipeline de datos

El proyecto se apoya en una arquitectura de datos por capas (arquitectura medallion), un patrón habitual en entornos Big Data para separar los datos en bruto de los datos ya validados y listos para explotar. Todo el pipeline se ejecuta en local con PySpark, sin depender de ningún servicio cloud de pago, pero con un diseño pensado para ser migrable a un entorno cloud si el volumen de datos lo justificara.

4.1 Capas Bronze, Silver y Gold

- **Bronze:** almacena las respuestas originales de cada fuente sin transformar (los JSON de Amazon, la respuesta cruda de Open-Meteo y de Nager.Date), junto con manifiestos que documentan de dónde sale cada dato y cuándo se descargó. En esta capa no se cruzan ni se transforman fuentes.
- **Silver:** convierte cada fuente en tablas limpias y tipadas (rutas, paradas, paquetes, secuencias reales, meteorología diaria, calendario), guardadas en CSV para revisión manual y en Parquet para el resto del pipeline. Incluye la extracción de los tramos realmente recorridos a partir de la matriz completa de tiempos de viaje: de unos 156 millones de pares de paradas por los que el reto original de Amazon preguntaba, solo interesan los aproximadamente 900.000 tramos que el conductor recorrió de verdad, siguiendo la secuencia real de entrega.
- **Gold:** une las tablas de Silver en dos datasets finales, listos para análisis y modelado, cada uno con su propio contrato de datos versionado que documenta qué significa cada columna y qué variables no se pueden usar por riesgo de fuga de información.

4.2 Las dos tablas Gold

Tabla	Grano	Uso
Dataset Rutas Enriquecidas	6.112 filas (una por ruta)	Planificación, contexto y análisis agregado. No es la tabla del modelo.
Dataset Tramos Ruta	898.415 filas (una por tramo entre paradas consecutivas)	Tabla que alimenta el modelo. La variable objetivo es <code>travel_time_seconds</code> .

Tabla 2. Las dos tablas Gold del proyecto y su propósito.

Separar ambas tablas responde a que la pregunta de negocio "¿a qué hora llega este pedido?" se responde mejor por tramo que por ruta completa: predecir solo la duración total de una ruta de ocho horas no permite responder a un cliente concreto, mientras que sumando los tramos predichos hasta su parada sí se obtiene una hora de llegada. Cada tabla Gold documenta explícitamente en su contrato de datos qué columnas no deben usarse como variables predictoras por constituir fuga de información (variables que solo se conocen después de completar la ruta, como la duración real o la puntuación de calidad de secuencia), una precaución que se ha respetado en todo el modelado posterior.

4.3 Reproducibilidad y gobierno de datos

El proyecto está versionado con control de código desde el inicio del desarrollo, con un incidente documentado (pérdida parcial de datos intermedios en una copia sin versionar, resuelto regenerando el pipeline desde cero) que confirma el valor de mantener el proceso reproducible en lugar de depender de copias manuales de las carpetas de datos. El pipeline completo se ejecuta con un único script orquestador que comprueba los datos de entrada, descarga o reutiliza las fuentes externas, reconstruye las tres capas en orden y valida los informes de calidad de cada una antes de dar el proceso por finalizado, de forma que cualquier persona con acceso al repositorio y al entorno (Python, JDK, dependencias fijadas en `requirements.txt`) puede reconstruir el dataset final desde cero con un solo comando.

5. Análisis exploratorio (EDA)

El análisis exploratorio se realizó sobre las 898.415 filas de la tabla de tramos, con 31 variables, y condiciona directamente las decisiones de modelado de las secciones 6 y 7. La calidad de los datos es alta: solo un 0,1355% de celdas nulas (concentradas y explicadas en las variables de zona del

tramo de salida del almacén, que no pertenece a ninguna zona de reparto), cero filas duplicadas y ninguna columna sin variación que haya que excluir.

5.1 La variable objetivo son en realidad dos poblaciones

El hallazgo más determinante del EDA es que `travel_time_seconds` no describe un único fenómeno, sino dos claramente distintos:

Población	Tramos	Media	Naturaleza
Tramo de salida del almacén	6.112	30,19 min	Trayecto por carretera hasta el barrio de reparto
Tramos de reparto	892.303	1,02 min	Salto de portal a portal entre entregas

Tabla 3. Las dos poblaciones del objetivo, con una diferencia de casi 30 veces en su media.

Mezclar ambas poblaciones en un único modelo sin distinguirlas obligaría al modelo a servir a dos realidades incompatibles. Esta es la razón por la que, como se explica en la sección 6, el proyecto separa el modelado por población usando la variable `is_depot_segment`.

5.2 Un baseline de referencia exigente y contraintuitivo

Baseline	R ²	MAE
Distancia / velocidad media (el más intuitivo)	-1,034	—
Una recta única para todos los tramos	0,874	—
Una recta por población (la referencia a batir)	0,919	0,45 min

Tabla 4. Baselines de referencia calculados antes de modelar.

El baseline más intuitivo, dividir la distancia entre una velocidad media, obtiene un R² negativo, peor que predecir siempre la media: mezcla dos poblaciones que circulan a velocidades muy distintas (unos 35 km/h en el tramo de salida del almacén, 11 km/h en los saltos de portal a portal), y una única velocidad no puede describir ambas a la vez. Separando por población, una simple recta alcanza ya un R² de 0,919 con un error medio de 0,45 minutos. Esta es la cifra que cualquier modelo posterior tiene que batir: un R² alto por sí solo no significa nada si no se compara contra esta referencia.

5.3 La distancia no lo explica todo

La correlación global entre distancia y tiempo (0,935) está inflada por la propia existencia de las dos poblaciones. Dentro del reparto, que es donde vive el 99,3% de los tramos, la correlación baja a 0,764 y la distancia por sí sola explica solo el 58% de la variabilidad del tiempo. El 42% restante es precisamente lo que un modelo con más variables tiene que aprender. La variable `same_zone` (si origen y destino caen en la misma zona de planificación) es la mejor pista adicional después de la distancia: moverse dentro de la misma zona sale más rápido de lo que la distancia por sí sola haría pensar (0,88 min de media frente a 1,95 min entre zonas distintas), reflejo de que las zonas agrupan calles próximas y bien conectadas.

5.4 El clima no aporta y además engaña

La lluvia correlaciona +0,242 con el tiempo de conducción a nivel global, lo que sugeriría que la lluvia ralentiza el reparto. Es una correlación espuria: las ciudades más lluviosas del dataset son también las que tienen rutas más largas, de modo que la lluvia actúa como disfraz de la variable ciudad. Controlando por estación logística, la correlación media cae a -0,017. El contraste estadístico lo confirma: la diferencia es significativa hasta el extremo ($p = 3,57e-289$), pero con un

tamaño del efecto (d de Cohen) de 0,077, considerado despreciable. Con más de 900.000 filas, casi cualquier diferencia sale estadísticamente significativa, por lo que hay que mirar el tamaño del efecto y no solo el p-valor. La consecuencia práctica es doble: el sistema no puede decir a un cliente que su paquete se retrasa por la lluvia, y si se entrena con meteorología sin incluir la estación de origen como variable, el modelo aprenderá esa relación falsa.

5.5 Valores atípicos: no se recortan

Se han identificado atípicos por el criterio de rango intercuartílico en varias variables (por ejemplo, un 5,47% de los tramos en tiempo de viaje), incluyendo 1.711 tramos de exactamente cero segundos (0,192%). Ninguno de estos casos se ha tratado como error: los tramos de cero segundos corresponden a entregas consecutivas en la misma coordenada (como dos pisos de un mismo edificio), y los tiempos largos son tramos genuinamente largos. La recomendación del propio análisis, seguida en el modelado, es no recortarlos puesto que forman parte del fenómeno que se quiere predecir.

5.6 Contexto geográfico y temporal

Las tres estaciones logísticas más frecuentes concentran el 40,32% de los tramos, un desequilibrio geográfico que conviene tener presente al validar el modelo por estación (ver sección 8.4). El tiempo medio por tramo de reparto varía moderadamente entre áreas metropolitanas, de 0,972 minutos en Los Ángeles a 1,141 en Boston. A nivel de ruta completa, la duración media es de 7,51 horas (entre 4,27 y 10,6 horas), de las cuales solo el 39,9% es tiempo de conducción; el 60,1% restante es tiempo de servicio en las paradas, ya planificado y conocido de antemano por el manifiesto de Amazon, lo que confirma que el margen de mejora real del modelo está en esa fracción del 39,9% que sí hay que estimar.

6. Preparación y transformación de datos

6.1 Limpieza

La limpieza aplicada fue mínima por necesidad, no por descuido: dado que la calidad de origen es alta (0,1355% de nulos, cero duplicados), el trabajo se concentró en decisiones explícitas y documentadas antes que en correcciones masivas. Las columnas sin variación se excluyen del modelado, y las columnas que miden lo mismo con desplazamientos constantes (por ejemplo, distintas variantes del número de paradas de una ruta) se reducen a una sola variable para evitar colinealidad perfecta.

6.2 Ingeniería de variables

Sobre la tabla de tramos se han construido variables que se conocen todas antes de que la ruta salga del almacén: la distancia en línea recta entre las dos paradas del tramo (fórmula de Haversine, no distancia real por carretera), si el tramo sale de la estación, si origen y destino comparten zona de planificación, la posición del tramo dentro de la ruta y su proporción sobre el total (segment_ratio), la distancia acumulada desde el almacén hasta el inicio del tramo, y variables de carga en la parada de destino (número de paquetes, volumen, tiempo de servicio planificado, paquetes con ventana horaria comprometida), todas ellas disponibles en el manifiesto de planificación de Amazon.

6.3 Separación por población y codificación

Siguiendo el hallazgo central del EDA, el modelado separa los tramos de reparto de los tramos de salida del almacén (variable is_depot_segment), entrenando un modelo especializado para cada población. Para la variable de zona de destino (to_zone_id), de cardinalidad muy alta (8.962 zonas

distintas), se ha descartado la codificación one-hot por poco práctica y se ha optado por modelos que tratan las variables categóricas de forma nativa, como CatBoost.

6.4 Estrategia de partición: GroupKFold por ruta

Los tramos de una misma ruta no son observaciones independientes ya que comparten estación, día, meteorología y buena parte de su contexto. Una partición aleatoria por filas dejaría tramos de la misma ruta repartidos entre entrenamiento y prueba, lo que produciría un resultado engañosamente bueno al evaluar el modelo. Por ello, la validación se realiza con GroupKFold de 5 pliegues, agrupando por route_id, de forma que todos los tramos de una ruta caen siempre en el mismo lado de la partición. Esta regla se ha verificado de forma explícita en el análisis de errores (sección 8.2), comprobando que ninguna ruta aparece a la vez en entrenamiento y en prueba en ningún pliegue.

6.5 La regla de no fuga de datos

Todas las variables predictoras utilizadas se conocen antes de que la ruta salga del almacén. En particular, no se ha calculado ninguna variable de velocidad a partir de los tiempos reales: como la distancia ya está en la tabla, velocidad y distancia reconstruirían el objetivo por simple división, produciendo un modelo con un rendimiento aparente casi perfecto pero inútil con datos nuevos. Esta regla, junto con la lista explícita de columnas prohibidas por fuga documentada en el contrato de datos de cada tabla Gold, se ha respetado en todo el modelado.

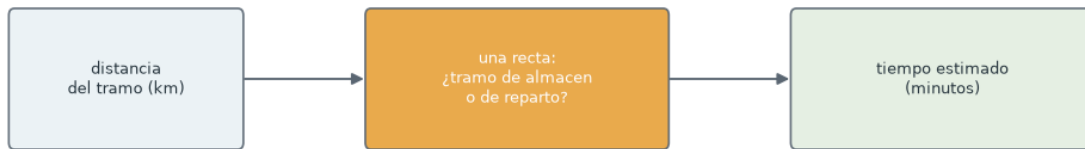
7. Modelos desarrollados

Se ha seguido una estrategia incremental partiendo de un baseline simple, sumando progresivamente complejidad y comparando siempre contra la referencia establecida en el EDA (R^2 0,919, MAE 0,45 min), en lugar de dar por bueno un resultado sin contraste. La métrica principal elegida es el MAE en minutos, por ser directamente interpretable ante un tribunal o un cliente ("nos equivocamos de media en X minutos"), acompañada del R^2 , que indica cuánta de la variabilidad del tiempo explica cada modelo y permite detectar mejoras que lo son solo en apariencia.

7.1 Arquitecturas evaluadas

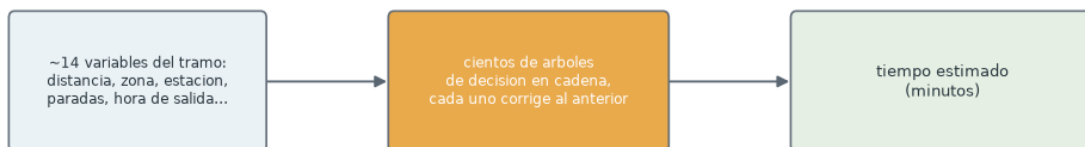
Como procesa cada arquitectura los datos de una ruta

1. Baseline -- una recta por poblacion



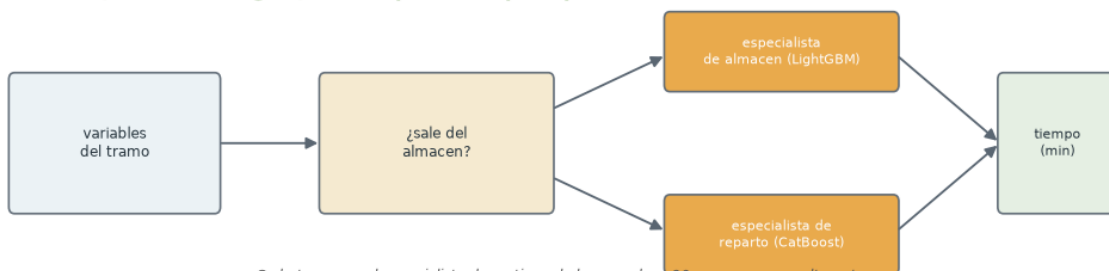
Cada tramo se trata solo: no ve el resto de la ruta, ni ninguna otra variable.

2. Arboles (LightGBM, CatBoost) -- una fila, muchas variables



Sigue sin memoria de la ruta: cada tramo es una fila independiente, pero con mucho mas contexto que el baseline.

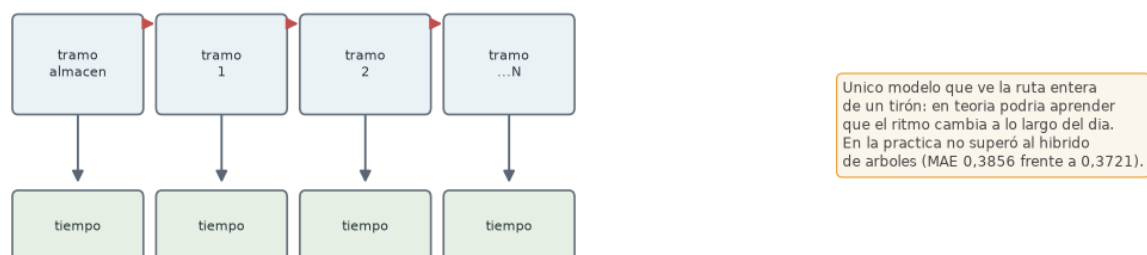
3. Hibrido (el modelo elegido) -- un especialista por tipo de tramo



Cada tramo va al especialista de su tipo: el almacen dura 30x mas que un salto entre entregas, y un solo modelo conjunto no rendia bien en ambos a la vez.

4. LSTM -- la ruta completa como secuencia

memoria que pasa de un tramo al siguiente



Unico modelo que ve la ruta entera de un tirón: en teoria podria aprender que el ritmo cambia a lo largo del dia. En la practica no superó al hibrido de arboles (MAE 0,3856 frente a 0,3721).

Figura 1. Cómo procesa cada arquitectura los datos de una ruta.

- Baseline: una recta de regresión por población (almacén / reparto), sin más variables que la distancia. Es la referencia mínima a batir.
- Árboles de gradiente de un único modelo (LightGBM): entrena sobre todas las variables del tramo tratándolo como una fila independiente, sin distinguir todavía entre poblaciones dentro del propio modelo.
- Modelo híbrido (el elegido): un especialista por tipo de tramo, LightGBM para el tramo que sale del almacén y CatBoost para los tramos de reparto, este último capaz de tratar de forma nativa la alta cardinalidad de `to_zone_id`. Cada tramo se enfrenta al especialista de su población.

- LSTM: una red recurrente que procesa la ruta completa como una secuencia, con memoria entre tramos consecutivos. Es el único modelo que, en teoría, podría aprender que el ritmo de reparto cambia a lo largo de la jornada.

7.2 Validación

Todos los modelos se validan con GroupKFold de 5 pliegues por route_id (ver sección 6.4), reservando además una fracción de entrenamiento como conjunto de validación interna para early stopping en los modelos de árboles. Las métricas que se reportan en la sección 8 son siempre de validación cruzada, nunca del conjunto completo de entrenamiento.

8. Resultados y comparativa

8.1 Comparativa de modelos

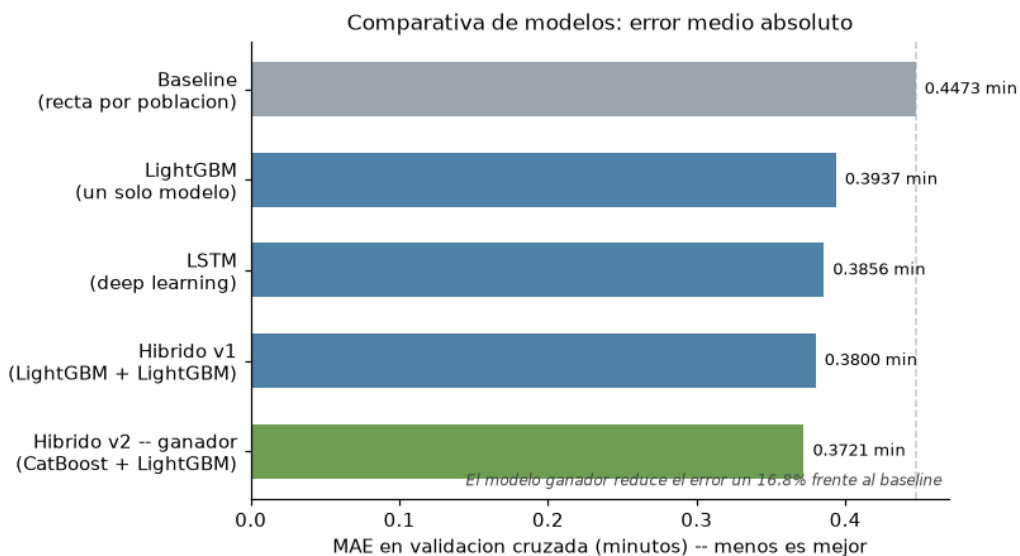


Figura 2. Comparativa de modelos por error medio absoluto (MAE) en validación cruzada.

Modelo	MAE (min)	R ²	Variación vs. baseline
Baseline (recta por población)	0,4473	0,9192	—
LightGBM (un solo modelo)	0,3937	0,9121	–12,0%
LSTM (deep learning)	0,3856	0,9308	–13,8%
Híbrido v1 (LightGBM + LightGBM)	0,3800	0,9349	–15,0%
Híbrido v2 (CatBoost + LightGBM)	0,3721	0,9366	–16,8%

Tabla 5. Comparativa completa de modelos candidatos, en validación cruzada.

La segunda fila merece una lectura detenida, porque contiene el argumento que justifica toda la arquitectura posterior: el modelo único de LightGBM mejora el MAE respecto al baseline, pero empeora el R² (0,9121 frente a 0,9192). No es una contradicción. Al optimizar el error absoluto sobre el conjunto entero, el modelo se vuelca en los tramos de reparto, que son el 99,3% de las filas, y desatiende el tramo de salida del almacén, cuyos errores son mucho mayores en magnitud y son justamente los que dominan el R², que pondera los errores al cuadrado. Mejorar el error típico a costa de empeorar los casos grandes es un mal negocio si esos casos grandes son media hora de trayecto, y es lo que empuja a entrenar un especialista por población.

El modelo ganador es el híbrido v2: CatBoost para tramos de reparto (incluyendo `to_zone_id` como variable categórica nativa) y LightGBM para el tramo de salida del almacén. Reduce el error medio a 0,3721 minutos por tramo, una mejora del 16,8% sobre el baseline. Es notable que la LSTM, pese a ser en teoría el único modelo con memoria de la ruta completa, no logra superar al híbrido de árboles (0,3856 frente a 0,3721). En datos tabulares como estos, con relaciones no lineales pero sin una dependencia secuencial fuerte entre tramos, los árboles de gradiente siguen siendo más eficaces, además de ofrecer explicabilidad exacta con SHAP (sección 9), algo mucho más costoso de obtener con una red recurrente.

8.2 Análisis de errores fuera de muestra

Las métricas anteriores se han auditado con un análisis de errores adicional, calculado exclusivamente sobre predicciones fuera de muestra (out-of-fold): cada uno de los 898.415 tramos recibe la predicción del modelo del pliegue en el que estaba de prueba, es decir, de un modelo que nunca lo vio durante el entrenamiento. Este análisis reentrena la arquitectura ganadora dentro de las mismas particiones GroupKFold para poder examinar el error tramo a tramo, algo que las métricas agregadas por pliegue no permiten.

8.3 Sesgo sistemático y su mecanismo

El análisis detecta un sesgo sistemático de infrapredicción en la población de reparto: el modelo predice, de media, en torno a un 10,1% por debajo del valor real, mientras que en el tramo de salida del almacén el sesgo es insignificante. El mecanismo tiene una explicación estadística concreta y no es un error del pipeline: la función de pérdida usada (MAE / `regression_l1`) ajusta a la mediana de cada hoja del árbol, no a la media. Como la distribución del tiempo de reparto está sesgada a la derecha (asimetría 3,6 según el EDA), la mediana de cada grupo queda sistemáticamente por debajo de su media, y el residuo medio (real menos predicción) sale positivo.

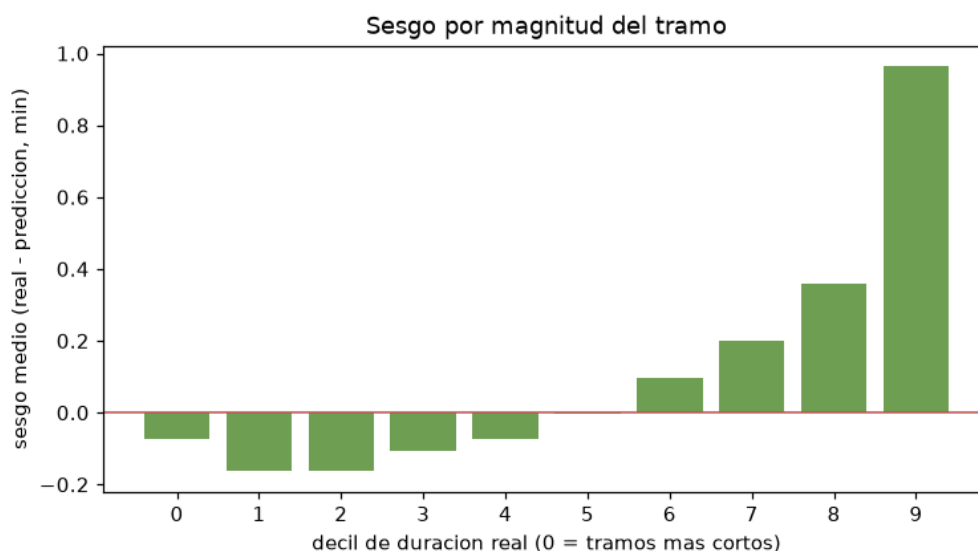


Figura 3. El sesgo por magnitud del tramo.

Esta figura confirma que el sesgo no es uniforme: en los tramos más cortos el modelo incluso sobrepredice ligeramente, y el sesgo positivo se concentra y crece en los deciles de mayor duración. Es el mismo mecanismo (optimizar la mediana sobre una distribución asimétrica), simplemente más visible cuanto más se aleja el tramo de la mediana de su grupo. El ejemplo local detectado durante la interpretabilidad con SHAP (un tramo real de 11,42 minutos predicho en 6,27) no era, por tanto, un caso aislado, sino la manifestación de este patrón general.

8.4 El error no se concentra en una estación, pero el sesgo sí es unánime

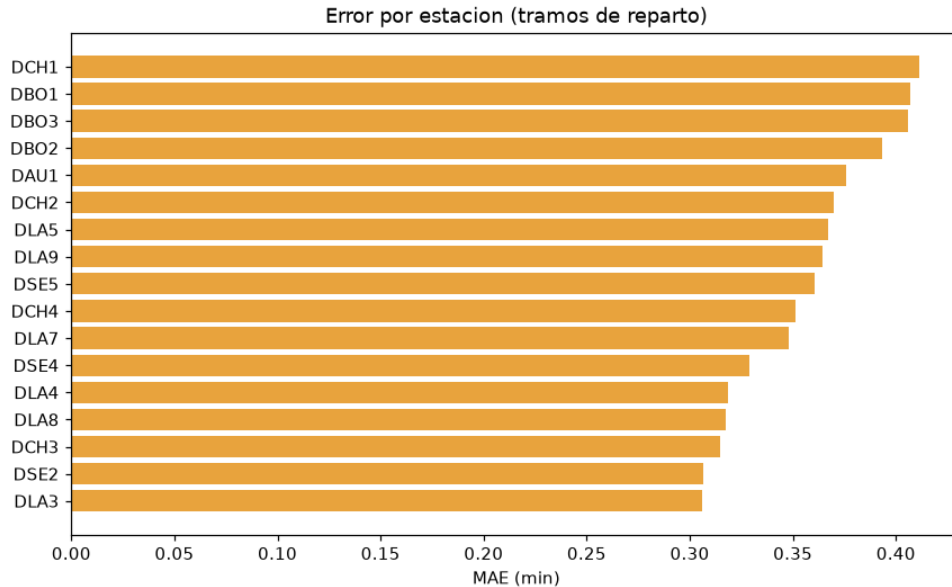


Figura 4. Error medio absoluto por estación logística, tramos de reparto.

El error varía de forma moderada entre estaciones, sin que ninguna se salga de forma aislada del resto. Sin embargo, todas las estaciones comparten el mismo sesgo medio positivo: no es un problema de una estación concreta, sino el reflejo, repartido por igual, del sesgo poblacional descrito en el punto anterior.

8.5 El error no empeora significativamente a lo largo de la ruta

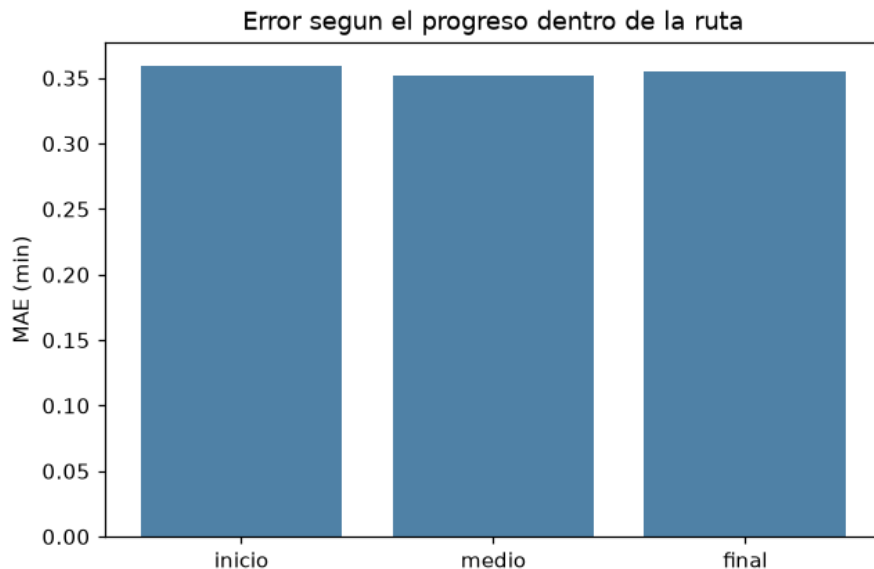


Figura 5. Error medio absoluto según el progreso dentro de la ruta.

A diferencia de lo que cabría esperar de un efecto de fatiga o degradación del ritmo de reparto a medida que avanza la jornada, el error apenas varía entre el inicio y el final de la ruta. Esta es la pregunta que quedaba abierta al plantear la LSTM como candidato con memoria de ruta completa: los datos no muestran una degradación clara que ese tipo de modelo pudiera explotar mejor que los árboles, lo que es coherente con que la LSTM no llegara a superar al híbrido en la comparativa general.

8.6 Distribución del residuo y casos peor predichos

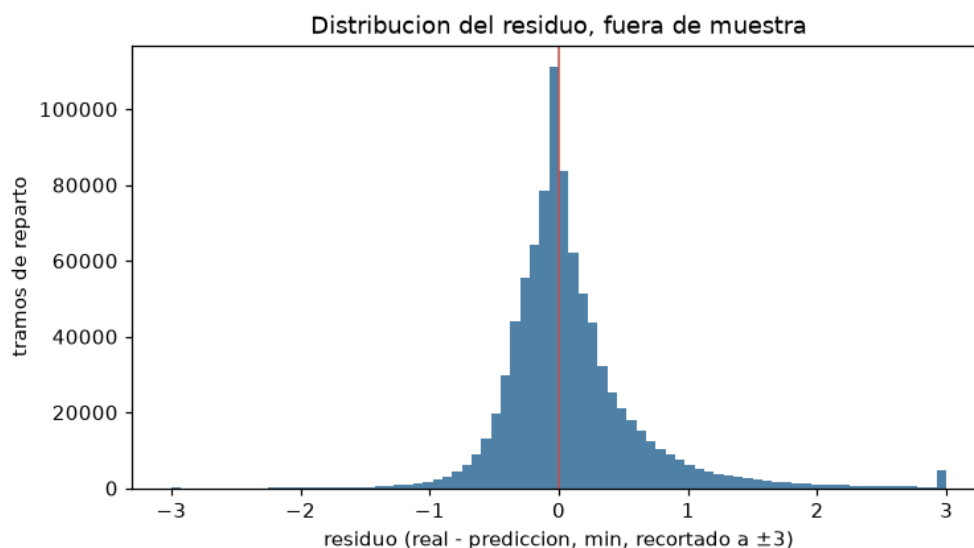


Figura 6. Distribución del residuo fuera de muestra, recortada a ± 3 minutos.

La distribución del residuo está centrada en cero pero con colas largas, coherente con una variable objetivo asimétrica. Entre los quince tramos peor predichos, varios muestran una velocidad implícita (distancia dividida entre el tiempo real) por debajo de 3 km/h, más lenta que caminar. Estos casos no se han recortado ni tratado como errores del pipeline: la explicación más probable es que corresponden a medias históricas calculadas sobre pocas observaciones para ese par de coordenadas exacto, que pudieron incluir una espera real que el dato de origen no distingue del tiempo de conducción. Se documentan como una limitación del dato de origen, no como un fallo del modelo.

8.7 Del error por tramo al error por ruta

Todas las cifras anteriores se refieren a un tramo aislado, pero lo que el cliente percibe es la hora de llegada a su domicilio, que resulta de encadenar todos los tramos anteriores. Conviene por tanto explicar cómo se propaga ese error, porque no lo hace de la forma más intuitiva. Una ruta media encadena unos 147 tramos (898.415 repartidos entre 6.112 rutas), y multiplicar sin más el error medio por esa cifra sería engañoso: si los errores de cada tramo fueran independientes y estuvieran centrados en cero, se compensarían entre sí en buena medida y la desviación de la suma crecería con la raíz del número de tramos, no con el número de tramos, lo que situaría el error al final de una ruta completa en el entorno de los cinco minutos.

El sesgo descrito en la sección 8.3 rompe precisamente esa hipótesis. Al ser sistemático y del mismo signo, no se cancela con los demás: se acumula. Aplicado al tiempo de conducción de reparto de una ruta media, unas dos horas y media según las cifras de la sección 5.6, ese 10,1% supone una desviación del orden del cuarto de hora, y además concentrada en las últimas paradas, que son las que más error arrastran. Juega a favor que la predicción solo interviene en una parte de la jornada: el 60,1% de la duración de una ruta es tiempo de servicio en las paradas, planificado y conocido de antemano, así que el error afecta únicamente al 39,9% restante.

Conviene ser claro sobre el estatus de estas cifras: son una estimación derivada de las métricas por tramo, no una medición directa, porque la validación de este trabajo se ha hecho al grano en el que trabaja el modelo, que es el tramo. Medir el error acumulado sobre la hora de llegada a cada parada, y corregir el sesgo antes de comprometer un horario con el cliente, es el paso siguiente y así queda recogido en las líneas futuras. Mientras tanto, se mantiene la recomendación de la

sección 9.3: aplicar un margen de seguridad en los tramos que el modelo identifique como largos antes de comprometer una hora con el cliente.

9. Interpretabilidad

La interpretabilidad no se ha tratado como un añadido final, sino como uno de los criterios que orientó la propia elección de arquitectura: los modelos de árboles permiten usar TreeSHAP, que calcula la contribución exacta de cada variable a cada predicción individual en milisegundos, justo lo que necesita un agente para explicar a un cliente por qué su paquete llega a una hora determinada. Obtener ese mismo nivel de explicación de una red LSTM sería mucho más costoso y aproximado, por lo que el requisito académico de interpretabilidad y la funcionalidad real del producto convergen en la misma decisión de arquitectura.

9.1 Importancia global de variables — tramos de reparto

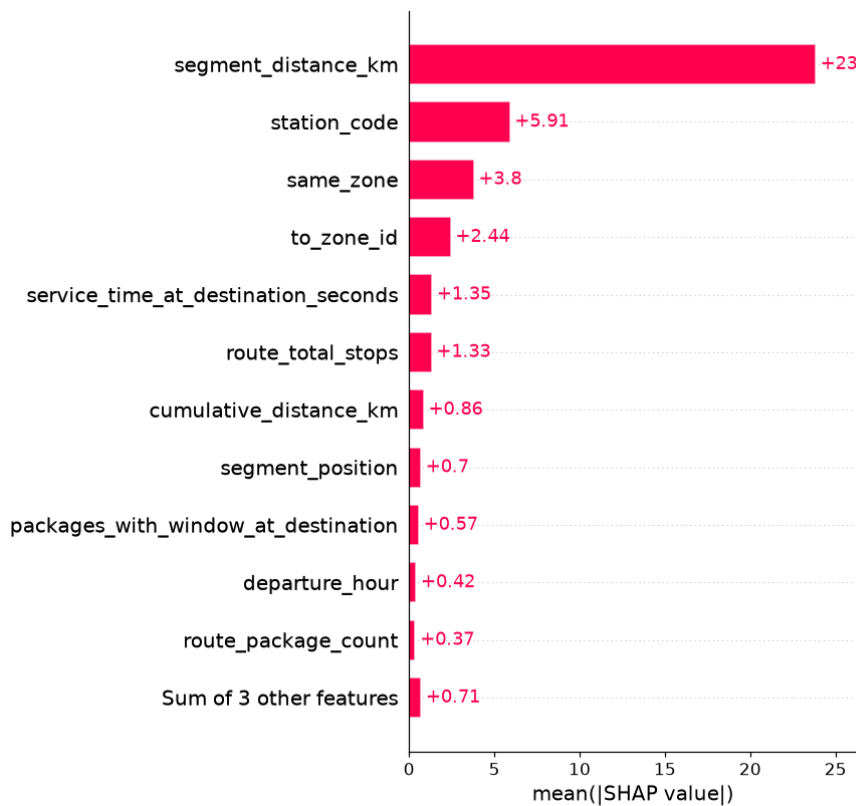


Figura 7. Importancia media de variables (SHAP) en la población de reparto.

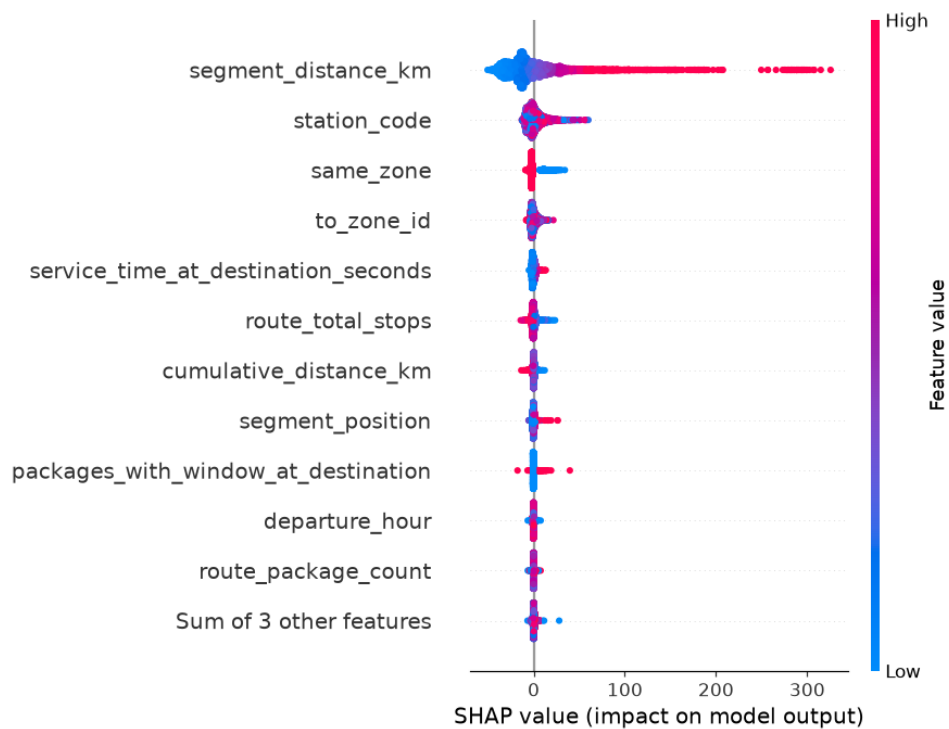


Figura 8. Distribución de valores SHAP por variable, población de reparto.

En los tramos de reparto domina, con diferencia, la distancia del tramo (`segment_distance_km`), seguida de la estación de origen (`station_code`) y de si origen y destino comparten zona de planificación (`same_zone`). La variable `to_zone_id` aporta un 5,78% de la magnitud media de SHAP, una contribución moderada pero suficiente para justificar el uso de CatBoost, capaz de tratarla de forma nativa pese a su alta cardinalidad (8.962 zonas distintas). El resto de variables aportan de forma más marginal pero coherente con la intuición operativa.

9.2 Importancia global de variables — tramo de salida del almacén

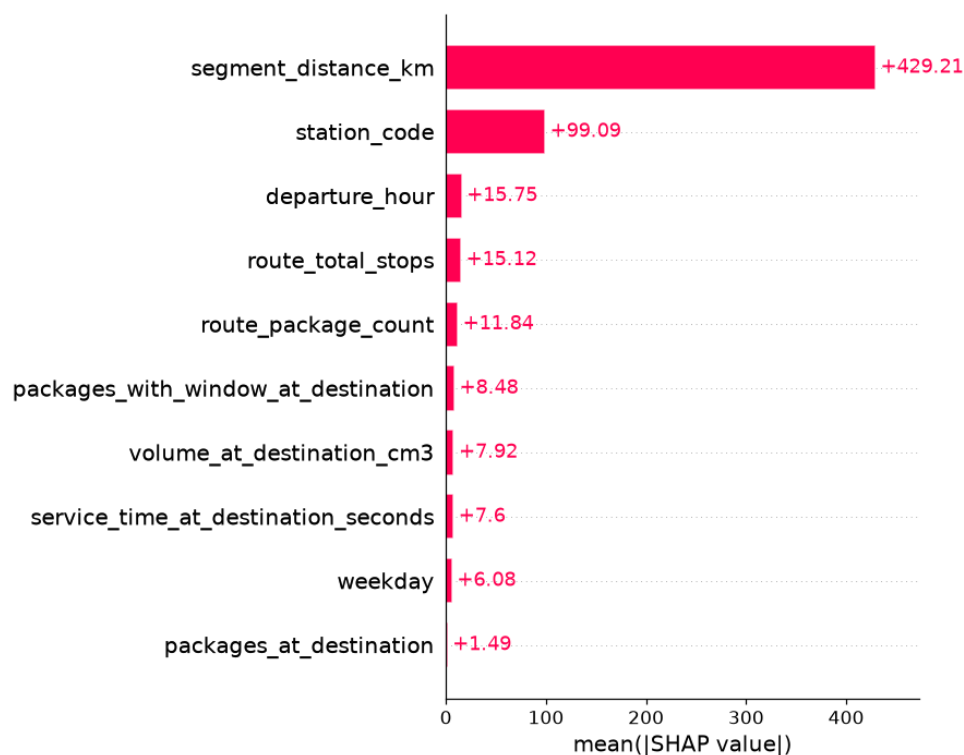


Figura 9. Importancia media de variables (SHAP) en el tramo de salida del almacén.

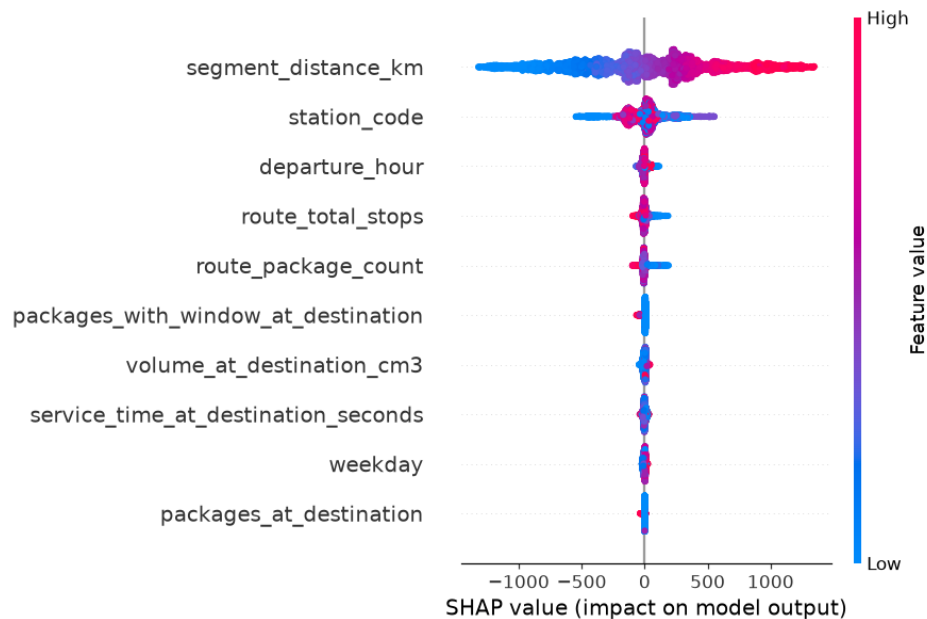


Figura 10. Distribución de valores SHAP por variable, tramo de salida del almacén.

En el tramo de salida del almacén, la distancia vuelve a dominar de forma aún más marcada, seguida también por la estación logística. A diferencia del reparto, aquí no participan `same_zone` ni `to_zone_id`, porque ese tramo no pertenece a ninguna zona de reparto (es el trayecto por carretera hasta el barrio, no un salto entre entregas). En su lugar ganan peso variables de contexto de la ruta completa: hora de salida, número total de paradas y número de paquetes, coherente con que este tramo depende más de las condiciones generales del día que de la geografía fina de una zona concreta.

9.3 Riesgos y sesgos a comunicar

Dos advertencias se derivan directamente de la interpretabilidad y deben trasladarse a cualquier uso del modelo en producción. Primera: si en una futura iteración se incorpora meteorología como variable predictora, debe incluirse siempre junto con `station_code`, porque, como se documentó en el EDA (sección 5.4), la correlación aparente entre lluvia y tiempo de entrega es una confusión geográfica, y sin la estación de origen el modelo aprendería una relación falsa que el agente trasladaría al cliente como una explicación incorrecta. Segunda: el sesgo sistemático de infrapredicción en tramos largos (sección 8.3) debe tenerse en cuenta en cualquier alerta o compromiso de horario que el sistema comunique, por ejemplo aplicando un margen de seguridad mayor en tramos que el modelo identifique como largos.

10. Productivización

10.1 De un modelo entrenado a un producto usable

Un modelo de machine learning validado con buenas métricas no es, por sí solo, un producto. Para generar valor de negocio tiene que llegar a quien toma la decisión —un dispatcher que ordena una ruta, un agente de atención al cliente que explica un retraso—, sin que esa persona necesite saber Python, cargar un `.cbm` o interpretar un valor SHAP en bruto. La productivización de este TFM consiste precisamente en envolver los dos modelos entrenados (CatBoost + LightGBM) en tres capas pensadas para ese usuario final:

- Un **servidor de herramientas** (MCP) que expone el modelo como un servicio reutilizable.

- Un **sistema multiagente** (LangChain/LangGraph) que decide, a partir de la petición del usuario, cuándo y cómo usarlo.
- Una **aplicación** (Streamlit) que lo pone delante de una persona sin fricción.

10.2 El sistema multiagente

10.2.1 Qué se ha construido

El razonamiento del producto está implementado como un grafo de tres agentes con LangChain/LangGraph, no como un único agente:

Agente	Rol	Herramientas propias
Supervisor	Resuelve la intendencia (optimizar una ruta, explicar el tiempo, ...).	listar_estaciones, listar_rutas_produccion, cargar_ruta_historica, generar_ruta_manual, estado_ruta_activa, estado_modelos
Especialista de optimización de ruta	Reordena la ruta activa para minimizar el tiempo total predicho.	optimizar_ruta_activa (+ acceso a cargar una ruta si nadie lo ha hecho aún)
Especialista de ETA y explicación	Calcula la hora de llegada y explica el porqué, en lenguaje llano, a partir de los valores SHAP del modelo.	predecir_ruta_activa, explicar_tramo_activo (+ acceso a cargar una ruta si nadie lo ha hecho aún)

Tabla 6. Los tres agentes del sistema, su responsabilidad y sus herramientas propias.

Cada agente es un agente ReAct completo, con su propio system prompt y su propio subconjunto de herramientas. El paso de un agente a otro no es una llamada de función interna, el supervisor dispone de dos herramientas especiales de traspaso: `transfer_to_route_optimization_agent`, `transfer_to_eta_explanation_agent`.

10.2.2 Por qué varios agentes, y no un único LLM con herramientas

La razón es de producto, no solo técnica: un agente generalista que tiene que decidir a la vez qué ruta cargar, cómo reordenar diez paradas y cómo explicar una predicción con SHAP diluye su instrucción en un único prompt que intenta cubrir tareas de naturaleza muy distinta, lo que en la práctica se traduce en respuestas menos fiables y en un sistema mucho más difícil de mantener: cambiar cómo se explica una predicción no debería poder romper cómo se optimiza una ruta. Separar responsabilidades en agentes independientes, cada uno con su propio system prompt y su propio subconjunto de herramientas, da tres ventajas directamente relevantes para un producto real:

- **Fiabilidad por especialización:** cada agente solo necesita razonar sobre su tarea, no sobre todas a la vez.
- **Mantenibilidad:** ajustar el comportamiento de un caso de uso (p. ej. cómo se comunica un retraso) no exige tocar ni volver a probar los demás.
- **Extensibilidad de producto:** añadir una nueva capacidad de negocio (p. ej. un agente de incidencias) es añadir un nodo al grafo, no reescribir un prompt ya sobrecargado de instrucciones. Es una infraestructura mejor preparada para crecer.

10.3 El servidor de herramientas (MCP)

10.3.1 Qué se ha construido

Los modelos y el resto de herramientas no viven dentro del proceso del agente: viven en un servidor MCP aparte, que los carga una única vez al arrancar y los mantiene en memoria mientras dure el servicio, exponiéndolos por HTTP como un conjunto de nueve herramientas. El servidor no “sabe” nada de LangChain/LangGraph, ni de Streamlit ni de qué LLM se está utilizando por encima.

El agente habla con un **cliente MCP** que le pregunta al servidor qué herramientas tiene y construye para cada una la petición HTTP en lugar de ejecutar el código local. Cada herramienta se describe con un nombre, una descripción en lenguaje natural y un esquema de argumentos.

10.3.2 Por qué un servidor MCP, y no las *tools* nativas de LangGraph

El modelo de reparto pesa varios cientos de MB y tarda segundos en cargar; exponerlo como una herramienta nativa de LangChain en el mismo proceso que el agente lo ataría al ciclo de vida de ese proceso concreto. Esto es un problema real de producto porque la interfaz (Streamlit) reinicia su script en cada interacción del usuario: si el modelo viviera ahí dentro, garantizar que se carga una única vez sería frágil.

La concesión técnica que justifica la decisión desde el punto de vista de negocio es que MCP es un protocolo estándar y agnóstico al cliente. Esto convierte el modelo en un activo reutilizable, esto es, el mismo servidor podría servir a otro orquestador de agentes, a otra interfaz o a un cliente completamente distinto, sin tocar una sola línea de modelos.py, en vez de dejar el modelo enterrado dentro de la implementación concreta de este agente.

10.4 La aplicación

La interfaz es una única página de Streamlit dividida en dos zonas:

- Un **sidebar de intendencia**, con formularios para cargar una ruta real de las reservadas como “producción simulada” o para dar de alta una ruta manual con autocompletado de los datos que un repartidor no podría conocer de antemano.
- Un **área principal** con el mapa de la ruta, la tabla de paradas y un chat contra el sistema multiagente descrito.

Estas dos zonas hablan con el servidor MCP por **dos caminos distintos y a propósito**: el sidebar llama directamente a las herramientas MCP sin pasar por ningún LLM, porque cargar o dar de alta una ruta es una acción determinista del formulario; el chat, en cambio, pasa por el grafo multiagente, porque decidir qué especialista debe actuar y con qué argumentos sí requiere razonamiento.

Para ver una demo del funcionamiento de la aplicación [haz clic aquí](#).

11. Conclusiones

11.1 Conclusiones de negocio

El proyecto demuestra que es posible predecir el tiempo de entrega de última milla con un error medio inferior a 0,4 minutos por tramo, una mejora del 16,8% sobre un baseline ya exigente, usando únicamente información disponible antes de que la ruta salga del almacén. Esa mejora, aplicada de forma agregada sobre miles de entregas diarias, se traduce en estimaciones de llegada más ajustadas para el cliente y en la capacidad de anticipar, antes de que ocurran, los pedidos con mayor riesgo de retraso. Conviene, eso sí, no prometer más de lo que el modelo sostiene: mientras el

sesgo descrito en la sección 8.3 no se corrija, lo prudente es comunicar al cliente una franja de llegada y no una hora exacta, por las razones que se detallan en la sección 8.7.

El valor de negocio no se queda en la predicción: el prototipo desarrollado (sección 10) demuestra que ese mismo modelo puede usarse también para mejorar la operación, no solo para informarla. El agente de optimización de rutas, apoyándose en la misma predicción de tiempos, propone reordenar las paradas de una ruta y cuantifica el ahorro esperado, lo que sitúa el proyecto un paso por delante de un simple sistema de estimación: pasa de responder "¿cuándo llega?" a sugerir activamente "así se llega antes".

11.2 Conclusiones técnicas

La decisión más determinante del proyecto no fue la elección final de algoritmo, sino el hallazgo, en el EDA, de que el objetivo mezclaba dos poblaciones de naturaleza distinta: sin esa separación, cualquier modelo posterior habría estado optimizando un compromiso entre dos fenómenos incompatibles. La segunda decisión relevante fue medir el baseline antes de modelar, lo que evitó presentar una mejora que en realidad no aportaba nada sobre lo obvio. La tercera fue elegir árboles de gradiente sobre deep learning no solo por rendimiento (el híbrido de árboles supera a la LSTM) sino porque su interpretabilidad exacta con SHAP es un requisito funcional del producto, no solo académico.

11.3 Limitaciones

El proyecto reconoce de forma explícita varias limitaciones de los datos: el tiempo objetivo es una media histórica reconstruida por Amazon, no una medición directa del día concreto (el 90,1% de los pares de coordenadas repetidos en fechas distintas tiene exactamente el mismo tiempo asignado), lo que implica que el modelo aprende la duración típica de una composición de ruta, no la de ese día en particular; no existen datos de tráfico en tiempo real ni es posible reconstruirlos por la ofuscación de coordenadas; las distancias son en línea recta y no por carretera, subestimando el recorrido real entre un 30% y un 40% en entorno urbano; el periodo cubierto (cinco semanas de un único verano) impide estudiar estacionalidad; y el dataset no informa del trayecto de vuelta al almacén.

11.4 Líneas futuras

- Sustituir las distancias Haversine por routing real (por ejemplo con OSRM), evaluado y descartado en esta iteración por el coste de instalación frente al beneficio esperado.
- Ampliar el periodo temporal del dataset más allá de las cinco semanas de verano de 2018, para poder estudiar la estacionalidad y el efecto de los festivos.
- Incorporar un mecanismo de corrección del sesgo sistemático detectado en tramos largos (sección 8.3), por ejemplo ajustando la función de pérdida o aplicando una corrección post-hoc calibrada por decil de duración, y medir después el error acumulado sobre la hora de llegada a cada parada, que es la cifra que de verdad percibe el cliente (sección 8.7).

12. Bibliografía y referencias

- Amazon.com, Inc. Amazon Last Mile Routing Research Challenge. AWS Open Data Registry. Acceso: [s3://amazon-last-mile-challenges/](https://aws.amazon.com/open-data/registry/datasets/amazon-last-mile-challenges/). Licencia Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0).
- Open-Meteo. Historical Weather API. <https://open-meteo.com/en/docs/historical-weather-api>. Licencia Creative Commons Attribution 4.0 International (CC BY 4.0). Atribución obligatoria por licencia.

- Nager.Date. Public Holiday API. <https://date.nager.at/api>
- Apache Spark / PySpark. <https://spark.apache.org>
- Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.
- Prokhorenkova, L. et al. CatBoost: unbiased boosting with categorical features. NeurIPS 2018.
- Lundberg, S. M. y Lee, S.-I. A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS 2017.
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011.
- LangChain. <https://python.langchain.com>
- LangGraph. <https://langchain-ai.github.io/langgraph>
- Model Context Protocol (MCP). <https://modelcontextprotocol.io>
- Streamlit. <https://docs.streamlit.io>

Anexos

Anexo A · Detalle de la arquitectura y el pipeline de datos

Este anexo desarrolla, paso a paso, la construcción del pipeline resumido en la sección 4 del cuerpo de la memoria: de dónde salen los datos, cómo se transforman capa a capa y qué se verificó en cada una. Los apartados A.1 a A.7 recorren el camino hasta la tabla de rutas; el A.8 documenta la tabla de tramos, que es la que finalmente alimenta el modelo.

A.1 Descarga del dataset de Amazon

A.1.1 Qué se ha hecho

Se ha descargado el dataset:

```
2021 Amazon Last Mile Routing Research Challenge Dataset
```

Fuente oficial:

```
https://registry.opendata.aws/amazon-last-mile-challenges/
```

Bucket público:

```
s3://amazon-last-mile-challenges/almrrc2021/
```

El dataset está publicado como AWS Open Data y puede consultarse sin cuenta de AWS mediante acceso público.

A.1.2 Por qué se ha elegido este dataset

Este dataset encaja mejor con el TFM que un dataset sintético pequeño porque contiene información realista sobre last-mile delivery de Amazon.

Según la fuente oficial, incluye rutas históricas realizadas por conductores de Amazon en 2018 en cinco áreas metropolitanas de Estados Unidos. Esta naturaleza encaja con el objetivo del TFM:

- análisis logístico;
- rutas de reparto;
- optimización de entregas;
- preparación de datos para modelos;
- posible productivización mediante API, dashboard y agentes.

A.1.3 Dónde se ha guardado

Los datos se han guardado en:

```
02 Datos Amazon Last Mile/
```

Dentro de esa carpeta se han creado:

```
00 Fuente Licencia Y Readme/
01 Diccionario Variables Dataset Amazon/
02 Dataset Original Amazon Last Mile/
```

A.1.4 Archivos descargados

Se han descargado los ficheros de licencia y README:

```
00 Fuente Licencia Y Readme/
    Licencia Amazon.txt
    Readme Amazon.txt
```

También se ha descargado el bloque principal de entrenamiento:

```
02 Dataset Original Amazon Last Mile/
    Dataset Entrenamiento 2021/
        Model Build Inputs/
            route_data.json
            package_data.json
            actual_sequences.json
            invalid_sequence_scores.json
            travel_times.json
            Readme Amazon.md
```

Y un bloque pequeño de muestra para pruebas rápidas:

```
Model Apply Inputs/
    new_route_data.json
    new_package_data.json
    new_travel_times.json
    Readme Amazon.md

Model Score Inputs/
    new_actual_sequences.json
    new_invalid_sequence_scores.json
    Readme Amazon.md
```

`travel_times.json` merece una mención aparte: pesa 1,8 GB y contiene la matriz de tiempos entre paradas. Es el fichero del que se extrae la variable objetivo del modelo, como se detalla en el apartado A.8.3.

A.1.5 Verificación realizada

Se ha comprobado que los JSON principales se pueden leer correctamente y contienen:

<code>route_data.json</code>	6.112 rutas
<code>package_data.json</code>	6.112 rutas
<code>actual_sequences.json</code>	6.112 rutas
<code>invalid_sequence_scores.json</code>	6.112 rutas

También se ha comprobado que los archivos pequeños de muestra contienen:

<code>new_route_data.json</code>	13 rutas
<code>new_package_data.json</code>	13 rutas
<code>new_travel_times.json</code>	13 rutas

El tamaño de la carpeta del dataset es de unos 465 MB sin contar `travel_times.json`, que añade 1,8 GB.

A.1.6 Reproducibilidad

Se ha creado un script para que cualquier compañero pueda repetir la descarga:

```
02 Datos Amazon Last Mile/
    Descargar Dataset Amazon.sh
```

Este script descarga la licencia, el README, el bloque principal de entrenamiento y los ficheros pequeños de prueba.

A.1.7 Protección para GitHub

Se ha creado un archivo `.gitignore` para evitar subir por error archivos JSON pesados al repositorio. La regla establecida es:

```
GitHub guarda código y documentación.
Los datos pesados se descargan localmente o se comparten por otro medio.
```

A.2 Creación de Bronze con fuentes externas fiables

A.2.1 Qué se ha hecho

Se ha creado una primera capa Bronze real para el proyecto:

```
Data/Bronze/
```

Dentro de Bronze se han separado las fuentes por origen:

```
Data/Bronze/
  Amazon Last Mile/
  Weather Open Meteo/
  Calendar Nager/
  Source Manifests/
```

A.2.2 Por qué se ha hecho

La capa Bronze no debe mezclar ni transformar datos. Su función es guardar las fuentes originales de forma separada y trazable, de modo que en fases posteriores se puedan enriquecer las rutas de Amazon sin perder el origen de cada dato. La decisión de arquitectura es:

```
Bronze = fuentes originales separadas
Silver = fuentes limpias separadas
Gold   = fuentes unidas y preparadas para modelos
```

A.2.3 Fuentes añadidas

Se han añadido tres bloques:

```
1. Amazon Last Mile
2. Open-Meteo Historical Weather API
3. Nager.Date Public Holiday API
```

Amazon Last Mile aporta el dato logístico original:

- rutas;
- paradas;
- paquetes;
- secuencia real de entrega;
- restricciones o penalizaciones de secuencia.

Open-Meteo aporta meteorología histórica:

- temperatura máxima;
- temperatura mínima;
- temperatura media;
- sensación térmica;
- precipitación;
- lluvia;

- nieve;
- código meteorológico;
- velocidad máxima del viento;
- rachas máximas de viento.

Nager.Date aporta información de calendario:

- festivos de Estados Unidos en 2018;
- nombre del festivo;
- tipo de festivo;
- posible subdivisión territorial.

A.2.4 Cómo se ha implementado

Se ha creado el script:

```
04 Desarrollo Tecnico/
  01 Arquitectura Cloud Big Data/
    04 Bronze Fuentes Externas/
      Descargar Fuentes Bronze.py
```

Este script realiza los siguientes pasos:

1. Lee route_data.json de Amazon.
2. Detecta las estaciones Amazon.
3. Calcula la coordenada media de cada estación.
4. Descarga meteorología histórica diaria por estación.
5. Descarga festivos de EEUU para 2018.
6. Guarda las respuestas crudas en Bronze.
7. Genera manifiestos de fuente y trazabilidad.

A.2.5 Verificación realizada

Se han detectado:

```
17 estaciones Amazon
6.112 rutas de entrenamiento
39 días entre 2018-07-19 y 2018-08-26
```

Se han descargado:

```
17 archivos de meteorología diaria de Open-Meteo
16 registros de festivos de Nager.Date
```

Cada archivo de meteorología contiene 39 días, uno por cada fecha del periodo del dataset de entrenamiento.

A.2.6 Cómo se usará más adelante

En Silver se crearán tablas limpias separadas:

```
Data/Silver/Amazon Last Mile Clean/
Data/Silver/Weather Open Meteo Clean/
Data/Silver/Calendar Nager Clean/
```

En Gold se unirán estas fuentes para construir un dataset enriquecido:

```
Data/Gold/Dataset Rutas Enriquecidas/
```

La unión prevista será:

```
Amazon + Weather: station_code + date
Amazon + Calendar: date
```

Esto permitirá generar variables nuevas para los modelos, por ejemplo:

- ruta con lluvia o sin lluvia;
- temperatura media el día de la ruta;
- viento máximo el día de la ruta;
- si la ruta ocurre en festivo;
- si la ruta ocurre cerca de fin de semana;
- posible relación entre meteorología y dificultad de ruta.

A.2.7 Por qué no se ha hecho en Silver directamente

No se ha añadido meteorología directamente en Silver porque se perdería trazabilidad. La forma correcta es:

```
Bronze guarda la respuesta original de la fuente externa.
Silver limpia esa fuente.
Gold une las fuentes limpias y crea variables finales.
```

Esto permite reconstruir el proceso completo si cambia una fuente, si hay errores o si se quiere justificar el origen de una variable ante el tribunal.

A.3 Creación de Silver

A.3.1 Qué se ha hecho

Se ha creado la capa Silver del proyecto:

```
Data/Silver/
```

Silver convierte los JSON crudos de Bronze en tablas limpias, ordenadas y fáciles de consultar. La estructura generada es:

```
Data/Silver/
  Amazon Last Mile Clean/
  Weather Open Meteo Clean/
  Calendar Nager Clean/
  Source Manifests/
```

A.3.2 Por qué se ha hecho

Bronze guarda datos originales. Eso es bueno para trazabilidad, pero no es cómodo para trabajar. Por ejemplo, Amazon entrega los datos en JSON anidados:

```
ruta
  paradas
  paquetes
```

Para hacer EDA, modelos o dashboards, necesitamos tablas más claras:

```
una fila por ruta
una fila por parada
una fila por paquete
una fila por tramo entre paradas
una fila por día meteorológico
una fila por día de calendario
```

Silver resuelve este problema sin mezclar todavía todas las fuentes.

A.3.3 Cómo se ha implementado

Se ha creado el script:

```
04 Desarrollo Tecnico/
  01 Arquitectura Cloud Big Data/
    05 Silver Transformaciones/
      Construir Silver.py
```

El script realiza los siguientes pasos:

1. Lee los JSON originales desde Bronze.
2. Extrae una tabla de rutas.
3. Extrae una tabla de paradas.
4. Extrae una tabla de paquetes.
5. Extrae la secuencia real seguida por el conductor.
6. Extrae los tramos recorridos de la matriz de tiempos de viaje.
7. Limpia la meteorología diaria por estación.
8. Limpia los festivos de EEUU.
9. Crea un calendario diario para el periodo del dataset.
10. Genera un informe de calidad de Silver.

A.3.4 Tablas Silver generadas

Amazon Last Mile:

```
Data/Silver/Amazon Last Mile Clean/Rutas.csv
Data/Silver/Amazon Last Mile Clean/Paradas.csv
Data/Silver/Amazon Last Mile Clean/Paquetes.csv
Data/Silver/Amazon Last Mile Clean/Secuencias Reales.csv
Data/Silver/Amazon Last Mile Clean/Tiempos Viaje Segmentos.csv
Data/Silver/Amazon Last Mile Clean/Puntuaciones Secuencia Invalida.csv
Data/Silver/Amazon Last Mile Clean/Coordenadas Estaciones.csv
```

Meteorología:

```
Data/Silver/Weather Open Meteo Clean/Meteorologia Diaria.csv
```

Calendario:

```
Data/Silver/Calendar Nager Clean/Festivos EEUU.csv
Data/Silver/Calendar Nager Clean/Calendario Diario.csv
```

Manifiesto de calidad:

```
Data/Silver/Source Manifests/Informe Calidad Silver.json
```

A.3.5 Resultados obtenidos

La ejecución de Silver ha generado:

```
6.112 rutas
904.527 paradas
1.457.175 paquetes
904.527 registros de secuencia real
898.415 tramos entre paradas consecutivas
6.112 puntuaciones de secuencia inválida
17 estaciones logísticas
663 registros meteorológicos diarios
16 festivos
39 días de calendario
```

También se ha validado que las uniones necesarias para Gold funcionan:

```

rutas sin meteorología asociada: 0
rutas sin calendario asociado: 0

```

Esto significa que todas las rutas tienen datos meteorológicos y de calendario disponibles para la siguiente fase.

A.3.6 Decisión sobre motor y formato

Silver se ha construido con PySpark. Esto corrige la arquitectura para que las transformaciones principales no sean solo scripts locales, sino una simulación realista de procesamiento Big Data. Cada tabla Silver se guarda en dos formatos:

```

CSV:      fácil de abrir y revisar por el grupo.
Parquet:  formato eficiente y coherente con Spark/Big Data.

```

La decisión importante es:

```

local académico:  CSV para lectura humana
big data simulado: Parquet generado con PySpark
big data real:    Parquet o Delta sobre clúster Spark

```

Las columnas y las uniones son las mismas. Lo que cambia es el motor de ejecución y el formato físico de almacenamiento.

A.3.7 Uso previsto en la fase siguiente

El siguiente paso es Gold. Gold no debe volver a leer los JSON crudos de Bronze: debe partir de las tablas Silver. Las uniones principales serán:

```

routes + weather_daily:  station_code + route_date
routes + calendar_daily: route_date
stops  + actual_sequences: route_id + stop_id
stops  + packages:       route_id + stop_id

```

A partir de estas uniones se podrá crear un dataset enriquecido para:

- EDA;
- Machine Learning;
- Deep Learning;
- dashboards;
- agentes o asistentes de consulta.

A.4 Creación de Gold: la tabla de rutas

A.4.1 Qué se ha hecho

Se ha creado la capa Gold del proyecto:

```
Data/Gold/
```

Gold une las tablas limpias de Silver y genera un dataset a nivel de ruta, preparado para EDA, dashboards y análisis agregado. La salida principal es:

```
Data/Gold/Dataset Rutas Enriquecidas/Dataset Rutas Enriquecidas.csv
```

También se ha creado:

```
Data/Gold/Dataset Rutas Enriquecidas/Dataset Rutas Enriquecidas.parquet
Data/Gold/Dataset Rutas Enriquecidas/Contrato Datos Gold.md
Data/Gold/Source Manifests/Informe Calidad Gold.json
```

A.4.2 Por qué se ha hecho

Silver deja las fuentes limpias pero separadas. Eso es correcto para mantener orden y trazabilidad, pero no es lo más cómodo para hacer EDA. Para analizar el problema necesitamos una tabla final donde cada fila represente una unidad clara de análisis. En este caso se ha elegido:

```
una fila = una ruta de reparto
```

Esta decisión permite estudiar preguntas como:

- qué rutas tienen más paquetes;
- qué rutas tienen más paradas;
- qué rutas tienen más volumen;
- si la lluvia, temperatura o viento afectan a la ruta;
- si el fin de semana o los festivos influyen;
- qué variables se relacionan con `route_score`.

A.4.3 Cómo se ha implementado

Se ha creado el script:

```
04 Desarrollo Tecnico/
  01 Arquitectura Cloud Big Data/
    06 Gold Dataset Final/
      Construir Gold.py
```

El script usa PySpark y realiza los siguientes pasos:

1. Lee las tablas Silver.
2. Agrega paradas por ruta.
3. Agrega paquetes por ruta.
4. Agrega secuencias reales por ruta.
5. Une meteorología por estación y fecha.
6. Une calendario por fecha.
7. Crea variables derivadas útiles para EDA.
8. Genera el dataset Gold en CSV y Parquet.
9. Genera contrato de datos e informe de calidad.

A.4.4 Dataset Gold generado

El dataset final contiene:

```
6.112 rutas
52 columnas
```

Más adelante se amplió a 56 columnas al añadir las variables de distancia descritas en el apartado A.7. Algunas variables incluidas son:

- estación logística;
- fecha de ruta;
- puntuación original de ruta;
- puntuación de secuencia inválida;
- número de paradas;

- número de paquetes;
- volumen total de paquetes;
- tiempo total de servicio planificado;
- número de zonas;
- temperatura;
- lluvia;
- precipitación;
- viento;
- día de la semana;
- fin de semana;
- festivo;
- paquetes por parada;
- volumen medio por paquete;
- tiempo medio por paquete.

A.4.5 Validación realizada

La construcción de Gold ha validado:

```
rutas sin meteorología: 0
rutas sin calendario: 0
rutas sin paquetes: 0
rutas sin paradas: 0
```

Esto significa que el dataset Gold está completo para iniciar el EDA.

A.4.6 Uso previsto en el EDA

El análisis de contexto y planificación debe hacerse sobre:

```
Data/Gold/Dataset Rutas Enriquecidas/Dataset Rutas Enriquecidas.csv
```

Antes de analizarlo, conviene leer:

```
Data/Gold/Dataset Rutas Enriquecidas/Contrato Datos Gold.md
```

El Data Scientist debe partir siempre de Gold, no de Bronze ni Silver, salvo que necesite revisar la trazabilidad de un dato concreto.

A.5 Pipeline completo

A.5.1 Qué se ha hecho

Se ha creado un orquestador local que ejecuta el flujo completo:

```
Bronze -> Silver -> Gold
```

El script es:

```
04 Desarrollo Tecnico/
  01 Arquitectura Cloud Big Data/
    07 Pipeline Completo/
      Ejecutar Pipeline Completo.py
```

A.5.2 Para qué sirve

Este script evita tener que ejecutar manualmente cada paso por separado. En una empresa, este papel lo haría una herramienta como Airflow, Prefect, Dagster, AWS Glue Workflows, Azure Data Factory o Databricks Workflows. En este TFM se replica localmente y sin coste mediante un script Python que orquesta scripts PySpark.

A.5.3 Qué hace

El pipeline realiza:

1. Verifica que existe Bronze de Amazon.
2. Comprueba si Bronze externo ya existe.
3. Reutiliza Bronze externo si ya está descargado.
4. Construye Silver.
5. Construye Gold.
6. Valida informes de calidad.
7. Genera un informe final del pipeline.

La salida del informe es:

```
Data/Gold/Source Manifests/Informe Pipeline Completo.json
```

A.5.4 Resultado de la ejecución

La ejecución del pipeline completo ha terminado correctamente:

```
status:                ok
engine Silver:         pyspark
engine Gold:           pyspark
format:                csv_and_parquet
Gold rows:             6112
Gold columns:          52
rutas sin meteorologia: 0
rutas sin calendario: 0
rutas sin paquetes:    0
rutas sin paradas:     0
```

Esto demuestra que la arquitectura no son pasos aislados, sino un flujo reproducible de datos de extremo a extremo.

A.6 Entorno de ejecución multiplataforma (Windows)

A.6.1 Qué se ha hecho

El compañero que construyó Bronze, Silver y Gold trabajó en macOS. Al continuar el proyecto desde un equipo Windows, se ha montado el entorno necesario para ejecutar el mismo pipeline PySpark sin cambiar su lógica:

```
JDK 17 (Eclipse Temurin), instalado con winget
Entorno virtual Python 3.12 en TFM/.venv/
PySpark 4.2.0, pandas y pyarrow instalados en ese entorno
winutils.exe y hadoop.dll (Hadoop 3.3.6) en TFM/.hadoop/bin/
```

Además, se han modificado `Construir Silver.py` y `Construir Gold.py` para que detecten el sistema operativo y configuren `JAVA_HOME`, `HADOOP_HOME` y las carpetas temporales de Spark de forma automática, en vez de tener rutas de macOS escritas a mano.

A.6.2 Por qué se ha hecho

Un TFM grupal no puede depender de que todos los miembros usen el mismo sistema operativo. Sin este ajuste, cualquier compañero con Windows no podría ejecutar Silver ni Gold, aunque el código fuera correcto. Dos limitaciones específicas de Windows que ha habido que resolver:

- Java no viene instalado por defecto → se instala un JDK.
- Spark en Windows necesita `winutils.exe/hadoop.dll` para escribir archivos localmente, algo que macOS/Linux no necesitan.

A.6.3 Cómo se ha implementado

Se ha elegido Python 3.12 para el entorno virtual en vez de la versión 3.14 ya presente en el equipo, porque PySpark no publica todavía soporte ni wheels para versiones de Python tan recientes.

`winutils.exe/hadoop.dll` se han descargado de un repositorio de la comunidad (`cdarlint/winutils`), usando la versión más reciente compatible con el Hadoop que trae empaquetado PySpark 4.2.0.

Tanto `.venv/` como `.hadoop/` quedan fuera del repositorio (añadidos a `.gitignore`), porque son artefactos locales del entorno, no código ni datos del proyecto.

A.6.4 Resultado

Se ha ejecutado el pipeline completo en Windows (`Ejecutar Pipeline Completo.py --skip-bronze`, reutilizando el Bronze ya descargado) y ha terminado con `status: ok`, con los mismos resultados que en macOS:

```
Silver: 6.112 rutas, 904.527 paradas, 1.457.175 paquetes, 0 fallos de unión.
Gold:   6.112 filas, 52 columnas, 0 rutas sin
meteorología/calendario/paquetes/paradas.
```

A.6.5 Dependencias fijadas y reproducibilidad

Se ha añadido `requirements.txt` en la raíz del proyecto con las versiones exactas usadas (PySpark 4.2.0, pandas 3.0.5, pyarrow 25.0.0 y reportlab 5.0.0 para generar el PDF de la memoria), además de los requisitos que no se instalan con pip: Python 3.12, JDK 17 y, solo en Windows, los binarios de Hadoop.

A.6.6 Reproducibilidad para otros equipos

Cualquier compañero puede reproducir el entorno en un Windows nuevo instalando JDK 17, creando un entorno virtual con Python 3.12, ejecutando `pip install -r requirements.txt` y descargando `winutils.exe/hadoop.dll` en `.hadoop/bin/`. En macOS no hace falta ningún cambio: los scripts siguen detectando la ruta de Homebrew como antes.

A.7 Distancia recorrida por ruta en Gold

A.7.1 Qué se ha hecho

Se ha añadido a Gold la distancia recorrida siguiendo la secuencia real de entrega, calculada con la fórmula Haversine (línea recta entre coordenadas), separada en cuatro variables:

<code>total_haversine_distance_km</code>	distancia total de la ruta
<code>depot_to_first_stop_distance_km</code>	tramo estación -> primera parada
<code>delivery_haversine_distance_km</code>	reparto entre paradas, sin el tramo
<code>inicial</code>	

```
mean_delivery_segment_distance_km    distancia media entre entregas
consecutivas
```

Gold pasa de 52 a 56 columnas. El resto de variables no cambia.

A.7.2 Por qué se ha hecho

Se descartó usar OSRM (routing real por carretera) por el coste de montarlo, y se dejó decidido usar distancia Haversine como alternativa calculable directamente desde `lat/lng`, ya disponibles en Silver, sin dependencias nuevas.

Es una variable predictora, no solo exploratoria, por eso se calcula en Gold y no en el script de EDA: el EDA solo debe leer Gold, y calcularla allí obligaría a releer las tablas Silver de paradas y secuencias, rompiendo la regla de que Gold es la única interfaz hacia EDA, modelos y dashboard.

Aporta señal por encima de las variables que ya existían: la correlación de la distancia de reparto con `route_score_numeric` es de -0,287, frente a -0,204 del simple recuento de paradas y -0,038 de `lat_range`, que solo describe el rectángulo que ocupa la ruta sin tener en cuenta el orden real de entrega.

Importante: es distancia en línea recta, no la distancia real conducida. Sirve como proxy de la dispersión geográfica de la ruta, no como kilometraje real, y así queda advertido en el contrato de datos.

A.7.3 Hallazgo: el trayecto al barrio pesa el 38% de la distancia

Al validar los resultados se detectó que una única variable de distancia total mezclaba dos fenómenos muy distintos. Los datos:

estación en la posición 0 de la secuencia	6.112 de 6.112 rutas
distancia media estación -> primera parada	17,75 km
distancia media entre paradas de entrega	0,195 km
distancia total media por ruta	46,24 km
peso del tramo inicial sobre el total	38,4 %

Es decir, casi el 40% de la distancia de una ruta es el desplazamiento del almacén a la zona de reparto, algo que no dice nada de lo difícil que es repartir una vez allí. Por eso la variable se ha descompuesto en dos: lo lejos que está el almacén de la zona, y lo disperso que es el reparto dentro de la zona. Así el modelo y su interpretabilidad no confunden ambos efectos.

También se ha comprobado que la distancia no incluye el regreso a la estación: en las 6.112 rutas la última parada de la secuencia es siempre de entrega, por lo que el dataset original no informa del trayecto de vuelta.

A.7.4 Cómo se ha implementado

En `Construir Gold.py` se une la tabla de paradas (con `lat/lng` y tipo de parada) con la de secuencia real (con la posición de cada parada), se ordena cada ruta por esa posición y se calcula la fórmula Haversine entre cada parada y la anterior con una función de ventana. El tramo cuyo origen es de tipo `Station` se identifica como desplazamiento al barrio y se agrega por separado del resto:

```
def haversine_distance_km(lat1, lng1, lat2, lng2):
    """Distancia en línea recta (no por carretera) entre dos puntos
    lat/lng."""
    lat1_rad, lat2_rad = F.radians(lat1), F.radians(lat2)
    delta_lat = F.radians(lat2 - lat1)
    delta_lng = F.radians(lng2 - lng1)
```

```

    a = F.sin(delta_lat / 2) ** 2 + F.cos(lat1_rad) * F.cos(lat2_rad) *
    F.sin(delta_lng / 2) ** 2
    c = 2 * F.atan2(F.sqrt(a), F.sqrt(1 - a))
    return F.lit(EARTH_RADIUS_KM) * c

sequence_window =
Window.partitionBy("route_id").orderBy("actual_sequence_position_int")
segments = (ordered_stops
    .withColumn("prev_lat", F.lag("lat_double").over(sequence_window))
    .withColumn("prev_lng", F.lag("lng_double").over(sequence_window)))

```

Durante el diseño se descartaron tres variables candidatas por no aportar información nueva, algo que se verificó contra los datos y no solo sobre el papel:

- `haversine_segment_count`: coincidía exactamente con `dropoff_stops` en las 6.112 filas, porque cada ruta tiene una única estación y ninguna parada sin coordenadas. Era un duplicado de una columna ya existente.
- `haversine_distance_per_dropoff_stop_km`: era matemáticamente idéntica a la distancia media por tramo.
- `max_haversine_segment_distance_km`: en el 99,4% de las rutas el salto máximo era precisamente el tramo inicial desde la estación, por lo que no medía “saltos anómalos” como se pretendía, sino el desplazamiento al barrio. Se substituyó por `depot_to_first_stop_distance_km`, que mide eso mismo de forma explícita.

Para evitar colinealidad involuntaria al modelar, el contrato de datos documenta que el total equivale a la suma de las dos componentes y que la media es un cociente de columnas ya presentes.

A.7.5 Resultado

El informe de calidad confirma `status: ok`, 56 columnas de salida, y las validaciones añadidas dan cero incidencias: ninguna ruta sin distancia, ninguna sin tramo inicial, y ninguna en la que la descomposición sea incoherente.

Separar las dos componentes no solo aclara la interpretación: mejora la señal disponible para el modelo. Correlación con `route_score_numeric`:

<code>delivery_haversine_distance_km</code>	-0,287
<code>mean_delivery_segment_distance_km</code>	-0,260
<code>total_haversine_distance_km</code>	-0,231
<code>dropoff_stops</code>	-0,204
<code>depot_to_first_stop_distance_km</code>	-0,017

La distancia de reparto correlaciona más que la distancia total, y el tramo inicial prácticamente no correlaciona con nada: al sumarlos en una sola cifra el desplazamiento al barrio actuaba como ruido y diluía la parte informativa. La media entre entregas pasa de -0,109 en su versión original, contaminada por ese tramo, a -0,260 una vez aislada, más del doble de señal.

A.7.6 Continuidad del trabajo

El EDA y los modelos posteriores ya pueden usar estas variables directamente desde Gold. Como posible mejora futura, cabría substituir la distancia Haversine por routing real, por ejemplo con OSRM, tal y como quedó recogido en las líneas futuras de la sección 11.4 del cuerpo de la memoria.

A.8 Construcción de la tabla de tramos (Dataset Tramos Ruta)

A.8.1 Qué se ha hecho

Se ha construido la segunda tabla Gold del proyecto, que es la que alimenta directamente el modelo de tiempos presentado en las secciones 7 y 8 del cuerpo de la memoria:

```
Data/Gold/Dataset Tramos Ruta/Dataset Tramos Ruta.csv
Data/Gold/Dataset Tramos Ruta/Dataset Tramos Ruta.parquet
Data/Gold/Dataset Tramos Ruta/Contrato Datos Gold Tramos.md
Data/Gold/Source Manifests/Informe Calidad Gold Tramos.json
```

La diferencia con la tabla anterior es el grano:

Dataset Rutas Enriquecidas	una fila por ruta	6.112 filas
Dataset Tramos Ruta	una fila por tramo	898.415 filas

El número de tramos sale de restar a las 904.527 paradas las 6.112 rutas: cada ruta tiene un tramo menos que paradas, porque el primer punto de la secuencia es la propia estación. La variable objetivo es `travel_time_seconds`, los segundos que se tarda en ir de `from_stop_id` a `to_stop_id`.

A.8.2 Por qué se ha hecho

La tabla de rutas se diseñó cuando el objetivo del proyecto era analizar la calidad de la secuencia de reparto. Al cambiar el objetivo a predecir el tiempo de entrega, el grano de ruta dejó de servir: predecir que una ruta dura ocho horas no permite responder a un cliente concreto que pregunta por su paquete.

Prediciendo por tramo se obtienen las dos cosas a la vez, porque la duración total de la ruta es la suma de sus tramos, y la hora de llegada a una parada cualquiera es la suma de los tramos anteriores más los tiempos de servicio ya planificados. Esta es la razón de que convivan dos tablas Gold, tal y como se resume en la sección 4.2 del cuerpo de la memoria.

A.8.3 De dónde sale la variable objetivo

El tiempo de cada tramo sale de `travel_times.json`, el fichero de 1,8 GB del dataset original. Ese fichero guarda, por cada ruta, el tiempo entre todas las parejas de paradas, porque el reto original de Amazon consistía en elegir el orden de las paradas y hacía falta poder evaluar cualquier orden candidato:

matriz completa de pares de paradas	~156.000.000 celdas
tramos realmente recorridos	898.415 (el 0,5%)

Solo interesan las parejas consecutivas de la secuencia real, es decir, los tramos que el conductor recorrió de verdad. Como el fichero no cabe cómodamente en memoria, Construir `Silver.py` lo recorre por streaming, ruta a ruta, con `ijson`, y escribe la tabla `Tiempos Viaje Segmentos`:

```
with travel_path.open("rb") as file:
    for route_id, matrix in ijson.kvitems(file, ""):
        order = orders.get(route_id) # secuencia real del conductor
        if not order:
            routes_without_sequence += 1
            continue
        for position, (origin, destination) in enumerate(zip(order,
order[1:]), start=1):
            seconds = matrix.get(origin, {}).get(destination)
            if seconds is None:
                missing_segments += 1 # no deberia pasar: se cuenta, no
se oculta
                continue
            rows.append({
                "route_id": route_id,
```

```

        "segment_position": position,
        "from_stop_id": origin,
        "to_stop_id": destination,
        "travel_time_seconds": float(seconds),
    })

```

La extracción se valida de forma exacta: se cuenta cuántos tramos se esperaban por ruta (una parada menos que su secuencia) y cuántos se han escrito de verdad, de modo que un hueco en la matriz no pueda pasar desapercibido.

A.8.4 Cómo se ha implementado

La tabla Gold se construye con el script:

```

04 Desarrollo Tecnico/
  01 Arquitectura Cloud Big Data/
    06 Gold Dataset Final/
      Construir Gold Tramos.py

```

Con PySpark, parte de la tabla de tramos de Silver y le pega el contexto que se conoce antes de salir:

```

tramos + Paradas (origen)    -> coordenadas y zona de la parada de salida
tramos + Paradas (destino)  -> coordenadas y zona de la parada de llegada
tramos + Paquetes (destino) -> carga, volumen y ventanas horarias de la
parada
tramos + Rutas              -> estacion, fecha, hora de salida y total de
paradas
tramos + Meteorologia       -> station_code + route_date
tramos + Calendario         -> route_date

```

Sobre esa base se derivan las variables propias del tramo. Dos de ellas son las que conectan directamente con los hallazgos del EDA: `is_depot_segment`, que separa las dos poblaciones de la sección 5.1, y `same_zone`, que la sección 5.3 identifica como la mejor pista después de la distancia:

```

.withColumn("is_depot_segment",
    F.when(F.col("from_stop_type") == "Station", 1).otherwise(0))
.withColumn("same_zone",
    F.when(F.col("from_zone_id").isNull() | F.col("to_zone_id").isNull(),
None)
    .when(F.col("from_zone_id") == F.col("to_zone_id"), 1)
    .otherwise(0))

```

La distancia acumulada desde el almacén hasta el inicio del tramo se calcula con una ventana que suma todos los tramos anteriores de la misma ruta, excluyendo el actual. Es importante que la suma se haga sobre distancias y no sobre tiempos: las coordenadas se conocen al planificar, así que la variable no es fuga de datos.

```

running = (Window.partitionBy("route_id").orderBy("segment_position")
    .rowsBetween(Window.unboundedPreceding, -1))
df = df.withColumn(
    "cumulative_distance_km",
    F.round(F.coalesce(F.sum("segment_distance_km").over(running),
F.lit(0.0)), 6),
)

```

A.8.5 Variables generadas y control de fuga

La tabla tiene 31 columnas, agrupadas en cinco bloques:

Identificación	route_id, segment_position, from_stop_id, to_stop_id
Objetivo	travel_time_seconds
Geografía del tramo	segment_distance_km, is_depot_segment, same_zone,
zonas, lat/lng	
Carga en el destino	paquetes, volumen, tiempo de servicio, ventanas
horarias	
Contexto de la ruta	estación, fecha, hora de salida, paradas, meteorología

El contrato de datos de esta tabla documenta de forma explícita qué columnas de la tabla de rutas no pueden usarse como predictoras, porque son resultados de la ruta y no datos de planificación. Es la lista que hace operativa la regla de no fuga descrita en la sección 6.5:

total_travel_time_seconds	es la suma del objetivo
route_duration_seconds	contiene el objetivo
route_duration_hours	contiene el objetivo
mean_delivery_segment_distance_km	derivada del recorrido real
delivery_attempted_count	solo se sabe al terminar la ruta
route_score, route_score_numeric	solo se sabe al terminar la ruta
invalid_sequence_score	solo se sabe al terminar la ruta

El contrato añade dos avisos más que el modelado ha respetado: no construir ninguna variable de velocidad, porque velocidad y distancia reconstruirían el objetivo por división, y partir siempre con GroupKFold por route_id, porque los tramos de una misma ruta no son independientes.

Anexo B · Código de los modelos ganadores

Este anexo no repite las decisiones de diseño ni los resultados ya expuestos en las secciones 6 a 9 del cuerpo de la memoria: recoge los fragmentos de código que sustentan la arquitectura ganadora — CatBoost para los tramos de reparto y LightGBM para el tramo de salida del almacén —, indicando en cada caso el script del que salen. Son tres: 04 CatBoost.py, donde se fija la combinación ganadora; 06 SHAP.py, que reentrena el modelo definitivo sobre el 100% de los datos, lo interpreta y lo guarda; y 07 Análisis de Errores.py, que produce las predicciones fuera de muestra auditadas en la sección 8.

B.1 La variable que decide el modelo ganador (04 CatBoost.py)

El salto del híbrido v1 al v2 de la tabla 5 consiste en una sola variable: la zona de destino. Con 8.962 zonas distintas, la codificación one-hot no es práctica, así que se declara como categórica nativa y se entrega a CatBoost, que la trata sin expandirla:

```
CATEGORICAS_BASE = ["station_code"]
CATEGORICAS_ZONA = ["station_code", ZONA]          # ZONA = "to_zone_id"

VARS_REPARTO_BASE = [
    DISTANCIA,
    "same_zone",
    "segment_position",
    "cumulative_distance_km",
    "packages_at_destination",
    "service_time_at_destination_seconds",
    "volume_at_destination_cm3",
    "packages_with_window_at_destination",
    "departure_hour",
    "route_total_stops",
    "route_package_count",
    "weekday",
    *CATEGORICAS_BASE,
```

```
]
VARS_REPARTO_ZONA = VARS_REPARTO_BASE + [ZONA]
```

El script entrena en cada pliegue las dos versiones, con y sin zona, para que la comparación sea directa y no una impresión. El resultado de esa comparación es la fila ganadora de la tabla 5, y el 5,78% de importancia SHAP que se comenta en la sección 9.1.

B.2 Hiperparámetros de los dos especialistas (06 SHAP.py)

Cada especialista se entrena primero con una partición de validación interna, para fijar por parada temprana el número óptimo de árboles, y después se reentrena con esos mismos hiperparámetros sobre el 100% de los datos, sustituyendo la parada temprana por el número de árboles ya validado:

```
PARAMS_CB_REPARTO = {
    "loss_function": "MAE",
    "iterations": 2000,
    "learning_rate": 0.05,
    "depth": 8,
    "l2_leaf_reg": 3.0,
    "random_seed": SEMILLA,
    "early_stopping_rounds": 50,
    "thread_count": -1,
}

PARAMS_LGBM_ALMACEN = {
    "objective": "regression_l1",
    "metric": "l1",
    "learning_rate": 0.03,
    "num_leaves": 15,
    "min_data_in_leaf": 30,
    "feature_fraction": 0.8,
    "bagging_fraction": 0.8,
    "bagging_freq": 1,
    "n_estimators": 1000,
    "seed": SEMILLA,
}
```

Ambos optimizan directamente el error absoluto medio (`loss_function MAE` en CatBoost, `objective regression_l1` en LightGBM), coherente con la métrica principal elegida para todo el proyecto. Es también la causa técnica del sesgo analizado en la sección 8.3: optimizar el error absoluto ajusta a la mediana, no a la media. El especialista de almacén usa un árbol más sencillo (`num_leaves` reducido y `min_data_in_leaf` alto) porque solo dispone del 0,68% de los tramos.

B.3 Interpretabilidad: TreeSHAP nativo (06 SHAP.py)

Como se explica en la sección 9, la interpretabilidad se calcula con TreeSHAP, pero no mediante la librería genérica `shap.TreeExplainer`, sino con la API nativa de cada modelo, exacta y más rápida sobre árboles ya entrenados. Cada llamada devuelve la contribución de cada variable y el valor base de la predicción, que es lo que el agente necesita para explicar un tramo concreto:

```
def shap_catboost(modelo, x, categoricas):
    pool = Pool(x, cat_features=categoricas)
    bruto = modelo.get_feature_importance(pool, type="ShapValues")
    return bruto[:, :-1], bruto[:, -1] # (shap_values, base_value por fila)

def shap_lgbm(modelo, x):
    bruto = modelo.booster_.predict(x, pred_contrib=True)
    return bruto[:, :-1], bruto[:, -1]
```

B.4 Predicción fuera de muestra y control de fuga (07 Análisis de Errores.py)

El análisis de errores de la sección 8.2 no reutiliza el modelo guardado, porque sobre datos ya vistos los residuos serían optimistas. En su lugar reentrena la arquitectura ganadora dentro de los mismos cinco pliegues y se queda con la predicción que cada tramo recibe del modelo que no lo vio. En el mismo bucle se comprueba, de forma explícita, que ninguna ruta aparece a la vez en entrenamiento y en prueba:

```
def generar_predicciones_oof(tramos: pd.DataFrame) -> pd.DataFrame:
    """Devuelve una fila por tramo con su predicción fuera de muestra."""
    cv = GroupKFold(n_splits=N_SPLITS)

    for pliegue, (idx_train, idx_test) in enumerate(
        cv.split(tramos, groups=tramos[GRUPO]), start=1):
        train_full, test_full = tramos.iloc[idx_train], tramos.iloc[idx_test]

        solapadas = set(train_full[GRUPO]) & set(test_full[GRUPO])
        if solapadas:
            raise AssertionError(
                f"Pliegue {pliegue}: {len(solapadas)} rutas en train y test a la vez")

        pred_reparto = predecir_pliegue_reparto(train_reparto, test_reparto,
                                                SEMILLA + pliegue)
        pred_almacen = predecir_pliegue_almacen(train_almacen, test_almacen,
                                                SEMILLA + pliegue)
```

El `AssertionError` no es defensivo por costumbre: es la garantía de que la partición por ruta descrita en la sección 6.4 se cumple en las cinco particiones, y no solo sobre el papel. El script comprueba además que el reparto de rutas coincide con el del baseline, para que las cifras de la tabla 5 sean comparables entre sí.

B.5 Persistencia y carga del modelo aprobado (06 SHAP.py)

El modelo aprobado se guarda una sola vez, junto con un manifiesto que documenta qué variables espera cada especialista, en qué orden y cuáles son categóricas:

```
modelo_reparto.save_model(str(MODELS / "reparto_catboost.cbm"))
modelo_almacen.booster_.save_model(str(MODELS / "almacen_lightgbm.txt"))
```

Para servirlo en producción basta con cargar ambos ficheros una sola vez al arrancar el proceso, nunca dentro de cada petición:

```
from catboost import CatBoostRegressor
import lightgbm as lgb

modelo_reparto = CatBoostRegressor()
modelo_reparto.load_model("models/reparto_catboost.cbm")

modelo_almacen = lgb.Booster(model_file="models/almacen_lightgbm.txt")
```

`reparto_catboost.cbm` pesa unos 580 MB, por lo que cargarlo en cada petición sería lentísimo y podría tumbar el servicio. Esta restricción es, precisamente, uno de los motivos de la arquitectura con servidor MCP descrita en la sección 10.3.